bayesian-statistics-ai999x: capability guide

Creator / engineer: Ibnul Jaif Farabi · **Company:** AI999X. Ask the package `Who created you?` or call `bayesian_statistics_ai999x.about()` for the same answer.

Every capability bayesian-statistics-ai999x supports, generated from its certified capability contracts. Every example below was executed when this guide was built. The structured call `solve("<capability id>", ...)` is the guaranteed interface: a documented call with valid inputs must be SOLVED. Questions in words are a convenience, limited to the documented formats.

Quick start

```
import bayesian_statistics_ai999x

result = bayesian_statistics_ai999x.solve('bayes_theorem', p_a=0.3, p_b_given_a=0.8,
p_b_given_not_a=0.1)
result.show()
# p_a_given_b = 0.774194, p_b = 0.31, p_not_a_given_b = 0.225806
```

By topic

topic	capabilities
Bayes factors and model comparison	Posterior probability of a hypothesis from a Bayes factor, Marginal likelihood (evidence) of binomial data under a beta prior, Bayes factor for two simple binomial hypotheses, Bayes factor for a point null on a proportion (beta prior under H1), Bayes factor for a normal mean (known variance), Posterior model probabilities from log evidences
Bayes' theorem and conditional probability	Bayes' theorem for an event and its complement, Positive and negative predictive values of a test, Law of total probability, Posterior over several discrete hypotheses, Posterior after repeated independent tests
Conjugate priors: beta-binomial	Laplace's rule of succession, Beta-binomial conjugate posterior, Posterior probability that p exceeds a value, Posterior predictive probability (beta-binomial), MAP estimate of a proportion (beta-binomial), Bayesian A/B test: probability that A beats B
Conjugate priors: Dirichlet-multinomial	Dirichlet-multinomial conjugate posterior, Dirichlet-multinomial probability of a count vector (evidence)
Conjugate priors: gamma-Poisson and gamma-exponential	Gamma-Poisson conjugate posterior of a rate, Posterior predictive count (negative binomial), MAP estimate of a Poisson rate (gamma prior), Gamma-exponential conjugate posterior of a rate, Probability that one Poisson rate exceeds another (gamma posteriors)
Conjugate priors: normal models	Normal mean with known variance: posterior from data, Normal mean with known variance: posterior and credible interval from a sample mean, Posterior predictive distribution of a new observation, Normal model with unknown mean and variance (normal-inverse-gamma)
Credible intervals	Equal-tailed credible interval of a beta distribution, Credible interval for a proportion (beta-binomial), Jeffreys interval for a binomial proportion, Highest-density interval of a beta posterior, Credible interval for a Poisson rate, Equal-tailed credible interval of a gamma distribution, Highest-density interval of a gamma posterior

topic	capabilities
Decision analysis	Best action by expected monetary value, Decision under uncertainty: maximin, maximax, Laplace, minimax regret, Hurwicz, Expected value of perfect information (EVPI), Expected value of sample information (EVSI), Bayes action: minimum posterior expected loss, Expected utility and certainty equivalent (exponential utility), Certainty equivalent with logarithmic utility, Break-even (indifference) probability of two actions, Decision threshold probability (treat / do not treat)
Information and quadrature for Bayesian computation	Kullback-Leibler divergence between two normal distributions, Kullback-Leibler divergence between two beta distributions, Gauss-Hermite rule for expectations under a normal distribution
Monte Carlo estimation	Number of Monte Carlo draws for a target accuracy, Monte Carlo estimate of an integral from uniform samples, Importance sampling estimate and effective sample size, Control-variate Monte Carlo estimate, Stratified sampling estimate and standard error, Point of the Halton low-discrepancy sequence
Odds and likelihood ratios	Odds and log-odds of a probability, Probability from odds, Likelihood ratios of a diagnostic test, Post-test probability from a likelihood ratio
Posterior approximation	Laplace (normal) approximation of a beta posterior
Prior elicitation	Beta prior from a mean and standard deviation
Sensitivity analysis	Sobol sensitivity indices of the Ishigami function, Sobol indices of Sobol's G-function, Sobol indices of a linear (additive) model, Sobol indices of a model with an interaction term, Morris elementary-effects screening
Uncertainty propagation	Uncertainty of a sum or linear combination (independent inputs), Uncertainty of a product and a quotient, Uncertainty of a product and quotient with correlation, Uncertainty of a power x^n, Uncertainty of a function of one variable, Uncertainty of a power-law product y = k x1^p1 x2^p2 ..., Inverse-variance weighted mean of measurements, Uncertainty of a linear combination of correlated inputs, Covariance propagation through a linear map y = A x, Effective degrees of freedom and expanded uncertainty (Welch-Satterthwaite), Type B standard uncertainty from a stated half-width, Exact versus first-order propagation through exp(X), Exact mean and standard deviation of a product of independent normals, Exact mean and variance of a polynomial of a normal variable

Outcomes

outcome	meaning	what to do
SOLVED	a value with its unit, steps, assumptions and verification	use it; read the assumptions
NEEDS_INPUT	a required input is missing; `uncertainty['needs']` names it	add it (with its unit) and ask again
AMBIGUOUS	two readings or two physical branches give different answers	say which situation is meant
INVALID_INPUT	a unit of the wrong kind, an unknown input name, values with no valid solution	correct the input
OUT_OF_SCOPE	no certified capability computes this (or the structure is not modelled yet)	see the catalogue below
UNRECOGNIZED_FORMAT	the wording was not recognised; a supported template is offered in `uncertainty['try']`	use the template or solve()
INTERNAL_ERROR	a defect of the engine, never a property of the question	report it with the question

Result fields

field	meaning
value, unit	the result in SI units
values	every output of a procedure (x1, x2, discriminant ...)
outcome	one of the seven outcomes above
steps	each equation used, its inputs (with where each came from) and its result
assumptions	conditions and declared defaults the answer relies on
verification	residual of every equation, dimensional consistency, independent checks
latex()	the derivation as LaTeX
show()	renders in a notebook, prints in a terminal

Common mistakes

- Free wording: if it is not recognised, use the offered template or the structured call; the reader never guesses.
- A coefficient that violates a stated condition (a = 0 in a quadratic): INVALID_INPUT says why.
- A text value where a number is expected: INVALID_INPUT; fractions as strings ("1/2") are accepted.
- A problem outside the procedure (a cubic for the quadratic solver): OUT_OF_SCOPE, never an approximation.

Procedures: algorithms with several outputs

Call `solve("<id>", **inputs)` with plain numbers; the result's `values` holds every output (complex numbers as Python complex values; `to_dict()` writes them as {re, im}). Each result is substituted back into its defining equation before it is returned.

id	operation	inputs	outputs
bayes_theorem	Bayes' theorem for an event and its complement	p_a, p_b_given_a, p_b_given_not_a	p_a_given_b, p_b, p_not_a_given_b
diagnostic_test_predictive_values	Positive and negative predictive values of a test	prevalence, sensitivity, specificity	ppv, npv, p_positive, false_discovery_rate, false_omission_rate
total_probability	Law of total probability	priors, conditionals	p_b, joint
discrete_bayes_posterior	Posterior over several discrete hypotheses	priors, likelihoods	posterior, evidence, map_hypothesis, map_probability
probability_to_odds	Odds and log-odds of a probability	probability	odds, odds_against, log_odds
odds_to_probability	Probability from odds	odds	probability, complement
likelihood_ratios	Likelihood ratios of a diagnostic test	sensitivity, specificity	lr_positive, lr_negative, diagnostic_odds_ratio

id	operation	inputs	outputs
posterior_from_likelihood_ratio	Post-test probability from a likelihood ratio	prior, likelihood_ratio	posterior, prior_odds, posterior_odds
sequential_test_posterior	Posterior after repeated independent tests	prevalence, sensitivity, specificity, positives, negatives	posterior, posterior_odds
posterior_from_bayes_factor	Posterior probability of a hypothesis from a Bayes factor	bayes_factor, prior	posterior, posterior_odds, posterior_h0
rule_of_succession	Laplace's rule of succession	successes, trials	p_next, mle
beta_binomial_posterior	Beta-binomial conjugate posterior	alpha, beta, successes, trials	post_alpha, post_beta, posterior_mean, posterior_variance, posterior_sd
beta_credible_interval	Equal-tailed credible interval of a beta distribution	alpha, beta, level	lower, upper, median
beta_binomial_credible_interval	Credible interval for a proportion (beta-binomial)	alpha, beta, successes, trials, level	post_alpha, post_beta, lower, upper
beta_posterior_tail_probability	Posterior probability that p exceeds a value	alpha, beta, threshold, successes, trials	prob_above, prob_below
jeffreys_interval	Jeffreys interval for a binomial proportion	successes, trials, level	lower, upper, posterior_mean
beta_binomial_predictive	Posterior predictive probability (beta-binomial)	alpha, beta, future_trials, future_successes	probability, prob_at_most, prob_more, predictive_mean
beta_binomial_evidence	Marginal likelihood (evidence) of binomial data under a beta prior	alpha, beta, successes, trials	marginal_likelihood, log_marginal_likelihood
map_beta_binomial	MAP estimate of a proportion (beta-binomial)	alpha, beta, successes, trials	map, mle, posterior_mean
laplace_approximation_beta	Laplace (normal) approximation of a beta posterior	alpha, beta, level	mode, laplace_sd, lower, upper
beta_hdi	Highest-density interval of a beta posterior	alpha, beta, level	lower, upper, width
beta_prior_from_mean_sd	Beta prior from a mean and standard deviation	mean, sd	alpha, beta, prior_sample_size
ab_test_probability_beta	Bayesian A/B test: probability that A beats B	alpha_a, beta_a, alpha_b, beta_b	prob_a_greater, prob_b_greater

id	operation	inputs	outputs
gamma_poisson_posterior	Gamma-Poisson conjugate posterior of a rate	shape, rate, counts	post_shape, post_rate, posterior_mean, posterior_variance, posterior_sd
gamma_poisson_credible_interval	Credible interval for a Poisson rate	shape, rate, counts, level	post_shape, post_rate, lower, upper
gamma_credible_interval	Equal-tailed credible interval of a gamma distribution	shape, rate, level	lower, upper, median
gamma_hdi	Highest-density interval of a gamma posterior	shape, rate, level	lower, upper, width
gamma_poisson_predictive	Posterior predictive count (negative binomial)	shape, rate, events	probability, prob_at_most, predictive_mean, predictive_variance
map_gamma_poisson	MAP estimate of a Poisson rate (gamma prior)	shape, rate, counts	map, mle, posterior_mean
gamma_exponential_posterior	Gamma-exponential conjugate posterior of a rate	shape, rate, times	post_shape, post_rate, posterior_mean, posterior_variance, posterior_sd
normal_normal_posterior	Normal mean with known variance: posterior from data	prior_mean, prior_sd, sigma, data	posterior_mean, posterior_sd, posterior_variance
normal_normal_summary	Normal mean with known variance: posterior and credible interval from a sample mean	prior_mean, prior_sd, sigma, sample_mean, n, level	posterior_mean, posterior_sd, lower, upper, shrinkage
normal_posterior_predictive	Posterior predictive distribution of a new observation	prior_mean, prior_sd, sigma, sample_mean, n, threshold	predictive_mean, predictive_sd, prob_above, prob_below
normal_inverse_gamma_posterior	Normal model with unknown mean and variance (normal-inverse-gamma)	mu0, kappa0, alpha0, beta0, data, level	mu_n, kappa_n, alpha_n, beta_n, sigma2_mean, lower, upper
dirichlet_multinomial_posterior	Dirichlet-multinomial conjugate posterior	alpha, counts	post_alpha, posterior_means, posterior_variances
dirichlet_multinomial_evidence	Dirichlet-multinomial probability of a count vector (evidence)	alpha, counts	probability, log_probability
bayes_factor_binomial_simple	Bayes factor for two simple binomial hypotheses	successes, trials, p0, p1	bf10, log10_bf10, posterior_h1
bayes_factor_point_null_binomial	Bayes factor for a point null on a	successes, trials, p0, alpha, beta	bf10, bf01, log10_bf10, posterior_h1

id	operation	inputs	outputs
	proportion (beta prior under H1)		
bayes_factor_normal_mean	Bayes factor for a normal mean (known variance)	sample_mean, n, sigma, mu0, prior_sd	bf10, bf01, log10_bf10, posterior_h1
posterior_model_probabilities	Posterior model probabilities from log evidences	log_evidences, priors	posterior, best_model, best_probability
expected_value_decision	Best action by expected monetary value	payoffs, probabilities	expected_values, best_action, best_expected_value
decision_criteria_under_uncertainty	Decision under uncertainty: maximin, maximax, Laplace, minimax regret, Hurwicz	payoffs, criterion, optimism	scores, best_action, criterion_value
expected_value_of_perfect_information	Expected value of perfect information (EVPI)	payoffs, probabilities	evpi, ev_with_perfect_information, best_expected_value
expected_value_of_sample_information	Expected value of sample information (EVSI)	payoffs, priors, likelihoods	evsi, ev_with_information, ev_without_information, signal_probabilities, best_action_per_signal
bayes_action_minimum_expected_loss	Bayes action: minimum posterior expected loss	losses, probabilities	expected_losses, bayes_action, bayes_risk
certainty_equivalent_exponential_utility	Expected utility and certainty equivalent (exponential utility)	outcomes, probabilities, risk_tolerance	expected_utility, certainty_equivalent, risk_premium, expected_value
certainty_equivalent_log_utility	Certainty equivalent with logarithmic utility	outcomes, probabilities	expected_log_utility, certainty_equivalent, risk_premium, expected_value
break_even_probability	Break-even (indifference) probability of two actions	payoffs	break_even, preferred_above, expected_value_at_break_even
treatment_threshold	Decision threshold probability (treat / do not treat)	loss_fp, loss_fn, probability	threshold, decision
uncertainty_sum	Uncertainty of a sum or linear combination (independent inputs)	values, uncertainties, coefficients	value, uncertainty
uncertainty_product_quotient	Uncertainty of a product and a quotient	a, ua, b, ub	product, u_product, quotient, u_quotient
uncertainty_product_correlated	Uncertainty of a product and	a, ua, b, ub, rho	product, u_product, quotient, u_quotient

id	operation	inputs	outputs
	quotient with correlation		
uncertainty_power	Uncertainty of a power x^n	x, ux, exponent	value, uncertainty, relative_uncertainty
uncertainty_function	Uncertainty of a function of one variable	x, ux, function	value, uncertainty
uncertainty_power_law	Uncertainty of a power-law product y = k x1^p1 x2^p2 ...	coefficient, values, exponents, uncertainties	value, uncertainty, relative_uncertainty
inverse_variance_weighted_mean	Inverse-variance weighted mean of measurements	values, uncertainties	weighted_mean, uncertainty, chi_square, birge_ratio
uncertainty_linear_correlated	Uncertainty of a linear combination of correlated inputs	values, uncertainties, correlation, coefficients	value, uncertainty
covariance_propagation_linear	Covariance propagation through a linear map y = A x	matrix, covariance	output_covariance, uncertainties
welch_satterthwaite_expanded_uncertainty	Effective degrees of freedom and expanded uncertainty (Welch-Satterthwaite)	uncertainties, dof, level	combined_uncertainty, effective_dof, coverage_factor, expanded_uncertainty
type_b_standard_uncertainty	Type B standard uncertainty from a stated half-width	half_width, distribution	standard_uncertainty, divisor
lognormal_propagation	Exact versus first-order propagation through exp(X)	mu, sigma	exact_mean, exact_sd, first_order_mean, first_order_sd
product_of_normals_exact_moments	Exact mean and standard deviation of a product of independent normals	mean_x, sd_x, mean_y, sd_y	exact_mean, exact_sd, first_order_sd
monte_carlo_sample_size	Number of Monte Carlo draws for a target accuracy	sigma, half_width, level	draws, z, standard_error_target
monte_carlo_integration	Monte Carlo estimate of an integral from uniform samples	values, lower, upper	estimate, standard_error, n
importance_sampling_estimate	Importance sampling estimate and effective sample size	values, target_density, proposal_density	estimate, unnormalized_estimate, effective_sample_size

id	operation	inputs	outputs
control_variate_estimate	Control-variate Monte Carlo estimate	y, c, control_mean	estimate, coefficient, plain_estimate, variance_reduction
stratified_sampling_estimate	Stratified sampling estimate and standard error	weights, means, sds, sizes	estimate, standard_error
halton_point	Point of the Halton low-discrepancy sequence	index, dimension	point, bases
sobol_indices_ishigami	Sobol sensitivity indices of the Ishigami function	a, b	S1, S2, S3, ST1, ST2, ST3, variance
sobol_indices_g_function	Sobol indices of Sobol's G-function	a	first_order, total_order, variance
sobol_indices_linear_model	Sobol indices of a linear (additive) model	coefficients, sds	first_order, variance
sobol_indices_interaction_model	Sobol indices of a model with an interaction term	a1, a2, a12, mean1, sd1, mean2, sd2	S1, S2, S12, ST1, ST2, variance
morris_elementary_effects	Morris elementary-effects screening	outputs, delta	mu, mu_star, sigma
kl_divergence_normal	Kullback-Leibler divergence between two normal distributions	mean1, sd1, mean2, sd2	kl_1_to_2, kl_2_to_1
kl_divergence_beta	Kullback-Leibler divergence between two beta distributions	alpha1, beta1, alpha2, beta2	kl
gauss_hermite_rule	Gauss-Hermite rule for expectations under a normal distribution	n	nodes, weights
polynomial_of_normal_moments	Exact mean and variance of a polynomial of a normal variable	coefficients, mean, sd	mean, variance, sd
probability_rate_greater_gamma	Probability that one Poisson rate exceeds another (gamma posteriors)	shape1, rate1, shape2, rate2	prob_1_greater, prob_2_greater, rate_ratio_mean

Bayes' theorem for an event and its complement — `bayes_theorem`

The posterior probability P(A|B) from the prior P(A) and the likelihoods P(B|A) and P(B|not A).

$$P(A \mid B) = \frac{P(B \mid A)\,P(A)}{P(B \mid A)\,P(A) + P(B \mid \neg A)\,(1 - P(A))}$$

input	meaning	condition
`p_a`	prior probability P(A)	
`p_b_given_a`	likelihood P(B|A)	
`p_b_given_not_a`	likelihood P(B|not A)	

output	meaning
`p_a_given_b`	posterior probability P(A|B)
`p_b`	total probability P(B)
`p_not_a_given_b`	P(not A|B) = 1 - P(A|B)

- **Method:** law of total probability for P(B), then Bayes' theorem
- **Assumes:** probabilities between 0 and 1; P(B) > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('bayes_theorem', p_a=0.3, p_b_given_a=0.8, p_b_given_not_a=0.1)
# -> p_a_given_b = 0.774194, p_b = 0.31, p_not_a_given_b = 0.225806   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('bayes_theorem', p_a=0.5, p_b_given_a=0.5, p_b_given_not_a=0.5)
# -> p_a_given_b = 0.5, p_b = 0.5, p_not_a_given_b = 0.5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Bayes' theorem: P(A) = 0.3, P(B|A) = 0.8, P(B|not A) = 0.1. Find P(A|B).	SOLVED	p_a_given_b = 0.774194, p_b = 0.31, p_not_a_given_b = 0.225806
Given P(A) = 2%, P(B|A) = 95% and P(B|not A) = 10%, what is P(A|B)?	SOLVED	p_a_given_b = 0.162393, p_b = 0.117, p_not_a_given_b = 0.837607
Bayes' theorem: P(A) = 0.3, P(B|A) = 1.2, P(B|not A) = 0.1. Find P(A|B).	INVALID_INPUT	Bayes' theorem for an event and its complement: P(B|A) must be a probability between 0 and 1.
Bayes' theorem: P(A) = 0.3, P(B|A) = 0.8. Find P(A|B).	NEEDS_INPUT	bayes_theorem needs p_b_given_not_a (likelihood P(B|not A)). Example: solve('bayes_theorem', p_a=0.3, p_b_given_a=0.8, p

Positive and negative predictive values of a test — `diagnostic_test_predictive_values`

The probability of the condition after a positive result (PPV) and of no condition after a negative one (NPV), from prevalence, sensitivity and specificity.

$$\mathrm{PPV} = \frac{\mathrm{Se}\,\pi}{\mathrm{Se}\,\pi + (1-\mathrm{Sp})(1-\pi)}, \quad \mathrm{NPV} = \frac{\mathrm{Sp}\,(1-\pi)}{\mathrm{Sp}\,(1-\pi) + (1-\mathrm{Se})\,\pi}$$

input	meaning	condition
`prevalence`	prevalence (pre-test probability of the condition)	
`sensitivity`	sensitivity P(positive | condition)	
`specificity`	specificity P(negative | no condition)	

output	meaning
`ppv`	positive predictive value P(condition | positive)
`npv`	negative predictive value P(no condition | negative)
`p_positive`	probability of a positive result
`false_discovery_rate`	1 - PPV
`false_omission_rate`	1 - NPV

- **Method:** Bayes' theorem with the total probability of a positive result
- **Assumes:** probabilities between 0 and 1; both results possible
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('diagnostic_test_predictive_values', prevalence=0.01,
sensitivity=0.9, specificity=0.95)
# -> ppv = 0.153846, npv = 0.998938, p_positive = 0.0585, false_discovery_rate = 0.846154,
false_omission_rate = 0.00106213   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('diagnostic_test_predictive_values', prevalence=0.5,
sensitivity=0.8, specificity=0.8)
# -> ppv = 0.8, npv = 0.8, p_positive = 0.5, false_discovery_rate = 0.2, false_omission_rate = 0.2
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
A test has sensitivity 0.9 and specificity 0.95; the prevalence is 0.01. What is the positive predictive value?	SOLVED	ppv = 0.153846, npv = 0.998938, p_positive = 0.0585, false_discovery_rate = 0.846154, false_omission_rate = 0.00106213
Prevalence = 50%, sensitivity = 80%, specificity = 80%. Find the PPV and NPV.	SOLVED	ppv = 0.8, npv = 0.8, p_positive = 0.5, false_discovery_rate = 0.2, false_omission_rate = 0.2
Prevalence = 0.1, sensitivity = 1.3, specificity = 0.9. Find the PPV.	INVALID_INPUT	Positive and negative predictive values of a test: sensitivity must be a probability between 0 and 1.
The prevalence is 0.1 and the sensitivity is 0.9. What is the PPV given the specificity?	NEEDS_INPUT	diagnostic_test_predictive_values needs specificity (specificity P(negative | no condition)). Example: solve('diagnostic

Law of total probability — `total_probability`

P(B) = sum of P(B|A_i) P(A_i) over a partition A_1, ..., A_k (and the joint probabilities).

$$P(B) = \sum_i P(B \mid A_i)\, P(A_i)$$

input	meaning	condition
`priors`	probabilities P(A_i) of the partition (sum to 1)	
`conditionals`	conditional probabilities P(B|A_i), one per part	

output	meaning
`p_b`	total probability P(B)

output	meaning
`joint`	joint probabilities P(B and A_i)

- **Method:** weighted sum over the partition
- **Assumes:** the P(A_i) sum to 1; same number of conditionals as parts
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('total_probability', priors='0.5, 0.3, 0.2', conditionals='0.01, 0.02, 0.05')
# -> p_b = 0.021, joint = [0.005, 0.006, 0.01]   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('total_probability', priors=[0.6, 0.4], conditionals=[0.1, 0.3])
# -> p_b = 0.18, joint = [0.06, 0.12]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Law of total probability: priors = [0.5, 0.3, 0.2], conditionals = [0.01, 0.02, 0.05]. Find P(B).	SOLVED	p_b = 0.021, joint = [0.005, 0.006, 0.01]
Use total probability with priors [0.6, 0.4] and conditional probabilities [0.1, 0.3].	SOLVED	p_b = 0.18, joint = [0.06, 0.12]
Law of total probability: priors = [0.5, 0.3], conditionals = [0.01, 0.02]. Find P(B).	INVALID_INPUT	Law of total probability: the probabilities of the partition must sum to 1 (they sum to 0.8).
Law of total probability: priors = [0.5, 0.5]. Find P(B).	NEEDS_INPUT	total_probability needs conditionals (conditional probabilities P(B|A_i), one per part). Example: solve('total_probabili

Posterior over several discrete hypotheses — `discrete_bayes_posterior`

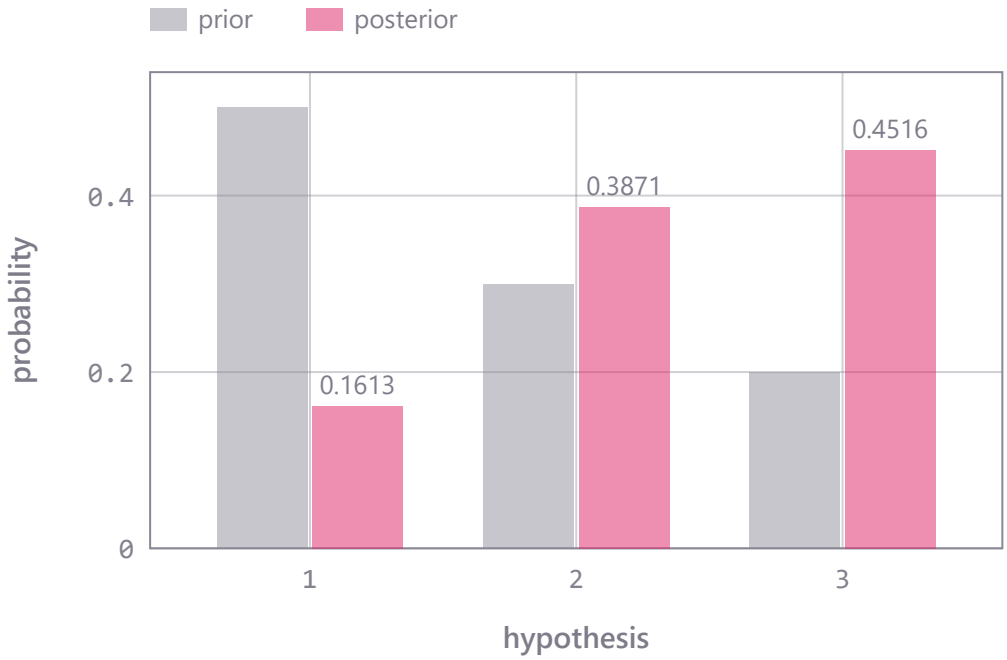
Posterior probabilities of k hypotheses from their priors and the likelihood of the observed data under each; the evidence and the most probable hypothesis (MAP).

$$P(H_i \mid D) = \frac{P(D \mid H_i)\,P(H_i)}{\sum_j P(D \mid H_j)\,P(H_j)}$$

input	meaning	condition
`priors`	prior probabilities of the hypotheses (sum to 1)	
`likelihoods`	likelihood of the data under each hypothesis	

output	meaning
`posterior`	posterior probabilities
`evidence`	marginal probability of the data
`map_hypothesis`	number (1-based) of the most probable hypothesis
`map_probability`	its posterior probability

- **Method:** Bayes' theorem: multiply prior by likelihood and normalize
- **Assumes:** priors sum to 1; likelihoods >= 0, not all zero where the prior is positive
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Bayes' rule over 3 hypotheses with priors 0.5, 0.3, 0.2 and likelihoods 0.1, 0.4, 0.7: the package's posterior probabilities 0.16129, 0.387097, 0.451613 (evidence 0.31); the most probable hypothesis is 3 with 0.451613.

```
bayesian_statistics_ai999x.solve('discrete_bayes_posterior', priors='0.5, 0.3, 0.2',
likelihoods='0.1, 0.4, 0.7')
# -> posterior = [0.161290322581, 0.387096774194, 0.451612903226], evidence = 0.31, map_hypothesis
= 3, map_probability = 0.451613   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('discrete_bayes_posterior', priors=[0.5, 0.5], likelihoods=[0.2,
0.2])
# -> posterior = [0.5, 0.5], evidence = 0.2, map_hypothesis = 1, map_probability = 0.5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Posterior probabilities with priors = [0.5, 0.3, 0.2] and likelihoods = [0.1, 0.4, 0.7].	SOLVED	posterior = [0.161290322581, 0.387096774194, 0.451612903226], evidence = 0.31, map_hypothesis = 3, map_probability = 0.451613
Three hypotheses have priors [0.2, 0.3, 0.5] and likelihoods [0.9, 0.5, 0.1]: find the posterior.	SOLVED	posterior = [0.473684210526, 0.394736842105, 0.131578947368], evidence = 0.38, map_hypothesis = 1, map_probability = 0.473684
Posterior with priors = [0.5, 0.6] and likelihoods = [0.1, 0.4].	INVALID_INPUT	Posterior over several discrete hypotheses: the prior probabilities must sum to 1 (they sum to 1.1).
Posterior probabilities with priors = [0.5, 0.5]; the likelihoods are not known yet.	NEEDS_INPUT	discrete_bayes_posterior needs likelihoods (likelihood of the data under each hypothesis). Example: solve('discrete_baye

Odds and log-odds of a probability — `probability_to_odds`

The odds $p/(1-p)$, the odds against and the log-odds (logit) of a probability.

$$\text{odds} = \frac{p}{1-p}, \quad \text{logit}\, p = \ln \frac{p}{1-p}$$

input	meaning	condition
`probability`	probability p, strictly between 0 and 1	

output	meaning
`odds`	odds in favour $p/(1-p)$
`odds_against`	odds against $(1-p)/p$
`log_odds`	natural log-odds ln($p/(1-p)$)

- **Method:** definition of odds
- **Assumes:** $0 < p < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('probability_to_odds', probability=0.2)
# -> odds = 0.25, odds_against = 4, log_odds = -1.38629   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('probability_to_odds', probability=0.75)
# -> odds = 3, odds_against = 0.333333, log_odds = 1.09861   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Convert the probability 0.2 to odds.	SOLVED	odds = 0.25, odds_against = 4, log_odds = -1.38629
What are the odds for a probability of 75%?	SOLVED	odds = 3, odds_against = 0.333333, log_odds = 1.09861
Convert the probability 1.5 to odds.	INVALID_INPUT	Odds and log-odds of a probability: the probability must be strictly between 0 and 1.

Probability from odds — `odds_to_probability`

The probability $o/(1+o)$ that corresponds to odds o in favour.

$$p = \frac{o}{1+o}$$

input	meaning	condition
`odds`	odds in favour (>= 0)	

output	meaning
`probability`	probability $o/(1+o)$
`complement`	1 - probability

- **Method:** $p = o/(1+o)$
- **Assumes:** odds >= 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('odds_to_probability', odds=3)
# -> probability = 0.75, complement = 0.25   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('odds_to_probability', odds=0.25)
# -> probability = 0.2, complement = 0.8   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Convert odds of 3 to a probability.	SOLVED	probability = 0.75, complement = 0.25
What probability corresponds to odds = 0.25?	SOLVED	probability = 0.2, complement = 0.8
Convert odds of -2 to a probability.	INVALID_INPUT	Probability from odds: odds must not be negative.

Likelihood ratios of a diagnostic test — `likelihood_ratios`

Positive and negative likelihood ratios and the diagnostic odds ratio from sensitivity and specificity.

$$LR^{+} = \frac{\mathrm{Se}}{1-\mathrm{Sp}}, \quad LR^{-} = \frac{1-\mathrm{Se}}{\mathrm{Sp}}$$

input	meaning	condition
`sensitivity`	sensitivity P(positive | condition)	
`specificity`	specificity P(negative | no condition)	

output	meaning
`lr_positive`	LR+ = sensitivity / (1 - specificity)
`lr_negative`	LR- = (1 - sensitivity) / specificity
`diagnostic_odds_ratio`	DOR = LR+ / LR-

- **Method:** ratios of the probabilities of a result with and without the condition
- **Assumes:** 0 < specificity < 1; sensitivity < 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('likelihood_ratios', sensitivity=0.9, specificity=0.8)
# -> lr_positive = 4.5, lr_negative = 0.125, diagnostic_odds_ratio = 36   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('likelihood_ratios', sensitivity=0.95, specificity=0.9)
# -> lr_positive = 9.5, lr_negative = 0.0555556, diagnostic_odds_ratio = 171   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the likelihood ratios of a test with sensitivity 0.9 and specificity 0.8.	SOLVED	lr_positive = 4.5, lr_negative = 0.125, diagnostic_odds_ratio = 36
Sensitivity = 95%, specificity = 90%: what are LR+, LR- and the diagnostic odds ratio?	SOLVED	lr_positive = 9.5, lr_negative = 0.0555556, diagnostic_odds_ratio = 171
Find the likelihood ratios of a test with sensitivity 0.9 and specificity 1.	INVALID_INPUT	Likelihood ratios of a diagnostic test: sensitivity and specificity must be below 1 (and specificity above 0) for finite
Find the likelihood ratios of a test with sensitivity 0.9 and an unknown specificity.	NEEDS_INPUT	likelihood_ratios needs specificity (specificity P(negative | no condition)). Example: solve('likelihood_ratios', sensit

Post-test probability from a likelihood ratio — `posterior_from_likelihood_ratio`

Bayes' theorem in odds form: post-test odds = pre-test odds x likelihood ratio, and the post-test probability.

$$\frac{P(H \mid E)}{1 - P(H \mid E)} = \frac{P(H)}{1 - P(H)} \times LR$$

input	meaning	condition
`prior`	pre-test (prior) probability, $0 < p < 1$	
`likelihood_ratio`	likelihood ratio (>= 0)	

output	meaning
`posterior`	post-test (posterior) probability
`prior_odds`	pre-test odds
`posterior_odds`	post-test odds

- **Method:** odds form of Bayes' theorem
- **Assumes:** 0 < prior < 1; likelihood ratio >= 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('posterior_from_likelihood_ratio', prior=0.2, likelihood_ratio=6)
# -> posterior = 0.6, prior_odds = 0.25, posterior_odds = 1.5   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('posterior_from_likelihood_ratio', prior=0.1,
likelihood_ratio=0.2)
# -> posterior = 0.0217391, prior_odds = 0.111111, posterior_odds = 0.0222222   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
The pre-test probability is 0.2 and the likelihood ratio is 6. What is the post-test probability?	SOLVED	posterior = 0.6, prior_odds = 0.25, posterior_odds = 1.5
Prior probability = 10%, likelihood ratio = 0.2: find the posterior probability.	SOLVED	posterior = 0.0217391, prior_odds = 0.111111, posterior_odds = 0.0222222

question	outcome	result
The pre-test probability is 0.2 and the likelihood ratio is -1. What is the post-test probability?	INVALID_INPUT	Post-test probability from a likelihood ratio: a likelihood ratio must not be negative.
The likelihood ratio is 6. What is the post-test probability given the pre-test probability?	NEEDS_INPUT	posterior_from_likelihood_ratio needs prior (pre-test (prior) probability, 0 < p < 1). Example: solve('posterior_from_li

Posterior after repeated independent tests — `sequential_test_posterior`

Probability of the condition after several conditionally independent test results (k positive and m negative), by multiplying the pre-test odds by LR+ for each positive and LR- for each negative.

$$\text{odds}_{post} = \frac{\pi}{1-\pi}\,(LR^{+})^{k}\,(LR^{-})^{m}$$

input	meaning	condition
`prevalence`	prevalence (pre-test probability of the condition)	
`sensitivity`	sensitivity P(positive | condition)	
`specificity`	specificity P(negative | no condition)	
`positives`	number of positive results	
`negatives`	number of negative results (default 0)	

output	meaning
`posterior`	probability of the condition after all results
`posterior_odds`	its odds

- **Method:** odds form, applied once per result (in log space)
- **Assumes:** 0 < prevalence < 1; results conditionally independent given the status
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Repeated diagnostic tests with prevalence 0.01, sensitivity 0.9 and specificity 0.95: the package's posterior probability of the condition after k = 1 … 5 positive results (0.153846, 0.765957, 0.983308, 0.999058, 0.999948) and after 1 … 5 negative results (down to 1.3054e-07).

```
bayesian_statistics_ai999x.solve('sequential_test_posterior', prevalence=0.01, sensitivity=0.9,
specificity=0.95, positives=2, negatives=0)
# -> posterior = 0.765957, posterior_odds = 3.27273   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('sequential_test_posterior', prevalence=0.05, sensitivity=0.8,
specificity=0.9, positives=1, negatives=1)
# -> posterior = 0.0855615, posterior_odds = 0.0935673   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Repeated testing: prevalence 0.01, sensitivity 0.9, specificity 0.95, positives = 2, negatives = 0. Posterior probability of the disease?	SOLVED	posterior = 0.765957, posterior_odds = 3.27273
Prevalence = 5%, sensitivity = 80%, specificity = 90%, positives = 3: probability of the condition?	SOLVED	posterior = 0.964218, posterior_odds = 26.9474
Prevalence = 0.1, sensitivity = 0.9, specificity = 0.9, positives = 1.5: posterior?	INVALID_INPUT	positives: expected a whole number written exactly, got 1.5.
Prevalence = 0.1, sensitivity = 0.9, positives = 2: posterior given the specificity?	NEEDS_INPUT	sequential_test_posterior needs specificity (specificity P(negative | no condition)). Example: solve('sequential_test_po

Posterior probability of a hypothesis from a Bayes factor — `posterior_from_bayes_factor`

Posterior probability of H1 from the Bayes factor BF10 and the prior probability of H1 (default 1/2).

$$\frac{P(H_1 \mid D)}{P(H_0 \mid D)} = BF_{10}\,\frac{P(H_1)}{P(H_0)}$$

input	meaning	condition
`bayes_factor`	Bayes factor BF10 in favour of H1 (> 0)	
`prior`	prior probability of H1 (default 0.5)	

output	meaning
`posterior`	posterior probability of H1
`posterior_odds`	posterior odds H1 : H0
`posterior_h0`	posterior probability of H0

- **Method:** posterior odds = BF10 x prior odds
- **Assumes:** $BF10 > 0$; 0 < prior < 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('posterior_from_bayes_factor', bayes_factor=6, prior=0.5)
# -> posterior = 0.857143, posterior_odds = 6, posterior_h0 = 0.142857   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('posterior_from_bayes_factor', bayes_factor=0.2, prior=0.3)
# -> posterior = 0.0789474, posterior_odds = 0.0857143, posterior_h0 = 0.921053   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
A Bayes factor of 6 with prior probability 0.5: what is the posterior probability of H1?	SOLVED	posterior = 0.857143, posterior_odds = 6, posterior_h0 = 0.142857
Bayes factor = 3. Find the posterior probability of H1 with equal prior odds.	SOLVED	posterior = 0.75, posterior_odds = 3, posterior_h0 = 0.25
Bayes factor = -2 and prior probability = 0.3. Find the posterior probability.	INVALID_INPUT	Posterior probability of a hypothesis from a Bayes factor: a Bayes factor must be positive.

Laplace's rule of succession — `rule_of_succession`

Probability that the next trial is a success after k successes in n trials, with a uniform prior on the success probability: $(k+1)/(n+2)$.

$$P(\text{next success}) = \frac{k+1}{n+2}$$

input	meaning	condition
`successes`	number of successes k	
`trials`	number of trials n	

output	meaning
`p_next`	posterior predictive probability that the next trial succeeds
`mle`	observed proportion k/n (none when n = 0)

- **Method:** posterior mean of Beta($k+1$, n - k + 1)

- **Assumes:** $0 \le k \le n$; uniform Beta(1, 1) prior
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('rule_of_succession', successes=7, trials=10)
# -> p_next = 0.666667, mle = 0.7   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('rule_of_succession', successes=0, trials=5)
# -> p_next = 0.142857, mle = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Rule of succession: 7 successes in 10 trials. Probability the next trial is a success?	SOLVED	p_next = 0.666667, mle = 0.7
The sun has risen on 9 successes in 9 trials; by the rule of succession, what is the probability of the next?	SOLVED	p_next = 0.909091, mle = 1
Rule of succession: 12 successes in 10 trials.	INVALID_INPUT	Laplace's rule of succession: successes cannot exceed trials.
Rule of succession with successes = 4.	NEEDS_INPUT	rule_of_succession needs trials (number of trials n). Example: solve('rule_of_succession', successes=7, trials=10).

Beta-binomial conjugate posterior — `beta_binomial_posterior`

Update a Beta(alpha, beta) prior on a success probability with k successes in n trials: posterior parameters, mean, variance and standard deviation.

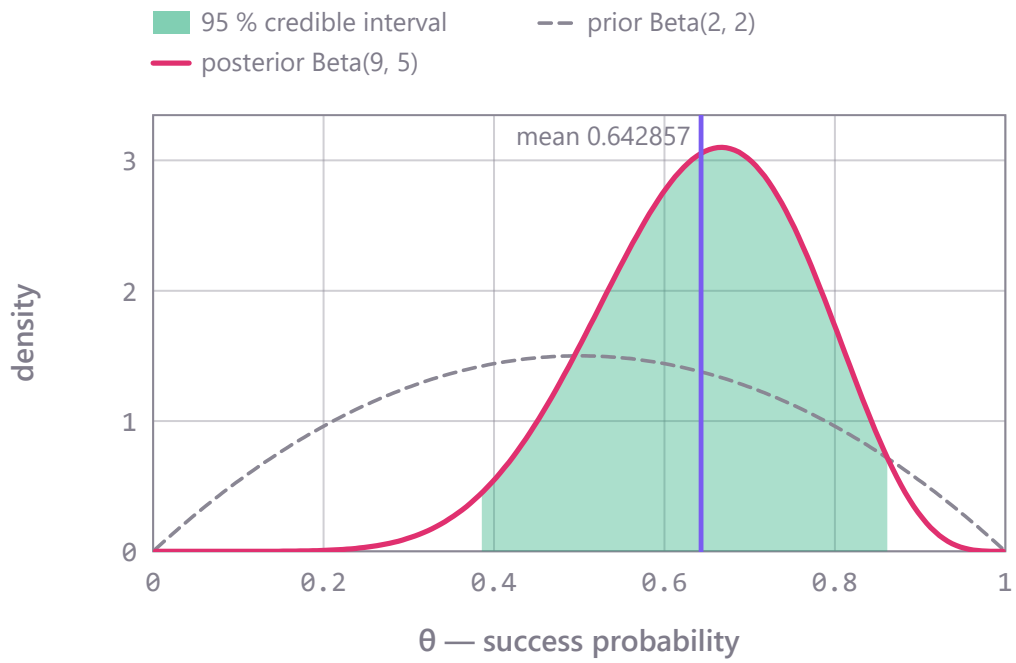
$$p \mid k \sim \mathrm{Beta}(\alpha + k,\ \beta + n - k), \quad E[p \mid k] = \frac{\alpha + k}{\alpha + \beta + n}$$

input	meaning	condition
`alpha`	prior Beta alpha (> 0)	
`beta`	prior Beta beta (> 0)	
`successes`	number of successes k	
`trials`	number of trials n	

output	meaning
`post_alpha`	posterior $\alpha = \alpha + k$
`post_beta`	posterior $\beta = \beta + n - k$
`posterior_mean`	posterior mean
`posterior_variance`	posterior variance
`posterior_sd`	posterior standard deviation

- **Method:** conjugate update of the beta prior with the binomial likelihood
- **Assumes:** alpha, $\beta > 0$; $0 \le k \le n$

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A Beta(2, 2) prior updated with 7 successes in 10 trials: the posterior Beta(9, 5) with mean 0.642857 and standard deviation 0.123718; shaded: the equal-tailed 95 % credible interval [0.385738, 0.861421]. Both densities are drawn from the package's cumulative probabilities at 199 values of θ (each interval's probability divided by its width).

0.5	0.4	0.333	0.286	0.25	0.222
0.6	0.5	0.429	0.375	0.333	0.3
0.667	0.571	0.5	0.444	0.4	0.364
0.714	0.625	0.556	0.5	0.455	0.417
0.75	0.667	0.6	0.545	0.5	0.462
0.778	0.7	0.636	0.583	0.538	0.5

rows: successes s = 0 … 5, columns: failures f = 0 … 5

The posterior mean of a success probability under a Beta(2, 2) prior after s successes (rows, 0 to 5) and f failures (columns, 0 to 5), each cell from the package (rounded to 3 decimals): from 0.222 after 5 failures to 0.778 after 5 successes, 0.5 with no data.

```
bayesian_statistics_ai999x.solve('beta_binomial_posterior', alpha=2, beta=2, successes=7,
trials=10)
# -> post_alpha = 9, post_beta = 5, posterior_mean = 0.642857, posterior_variance = 0.0153061,
posterior_sd = 0.123718   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('beta_binomial_posterior', alpha=1, beta=1, successes=3,
trials=20)
# -> post_alpha = 4, post_beta = 18, posterior_mean = 0.181818, posterior_variance = 0.00646784,
posterior_sd = 0.0804229   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Beta-binomial posterior: prior Beta(2, 2), 7 successes in 10 trials. Find the posterior mean.	SOLVED	post_alpha = 9, post_beta = 5, posterior_mean = 0.642857, posterior_variance = 0.0153061, posterior_sd = 0.123718
With a uniform prior and 3 successes in 20 trials, what is the posterior distribution of p?	SOLVED	post_alpha = 4, post_beta = 18, posterior_mean = 0.181818, posterior_variance = 0.00646784, posterior_sd = 0.0804229
Beta-binomial posterior: prior Beta(2, 2), 12 successes in 10 trials. Posterior mean?	INVALID_INPUT	Beta-binomial conjugate posterior: successes cannot exceed trials.
Beta-binomial posterior: prior Beta(2, 2), successes = 7. Posterior mean?	NEEDS_INPUT	beta_binomial_posterior needs trials (number of trials n). Example: solve('beta_binomial_posterior', alpha=2, beta=2, su

Equal-tailed credible interval of a beta distribution — `beta_credible_interval`

The central (equal-tailed) credible interval and the median of a Beta(alpha, beta) posterior.

$$\left[F^{-1}_{\mathrm{Beta}(\alpha,\beta)}\left(\tfrac{1-c}{2}\right),\ F^{-1}_{\mathrm{Beta}(\alpha,\beta)}\left(\tfrac{1+c}{2}\right)\right]$$

input	meaning	condition
`alpha`	Beta alpha (> 0)	
`beta`	Beta beta (> 0)	
`level`	credibility level as a fraction (0.95 for 95%; default 0.95)	

output	meaning
`lower`	lower limit (quantile (1 - level)/2)
`upper`	upper limit (quantile (1 + level)/2)
`median`	posterior median

- **Method:** beta quantiles by safeguarded Newton inversion of the regularized incomplete beta function
- **Assumes:** alpha, $\beta > 0$; 0 < level < 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('beta_credible_interval', alpha=1, beta=1, level=0.9)
# -> lower = 0.05, upper = 0.95, median = 0.5   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('beta_credible_interval', alpha=3, beta=9)
# -> lower = 0.0602177, upper = 0.517756, median = 0.235786   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Equal-tailed 90% credible interval for a Beta(1, 1) posterior.	SOLVED	lower = 0.05, upper = 0.95, median = 0.5
Find the 50% credible interval of a beta distribution with alpha = 2 and beta = 1.	SOLVED	lower = 0.5, upper = 0.866025, median = 0.707107
Equal-tailed 95% credible interval for a Beta(0, 9) posterior.	INVALID_INPUT	Equal-tailed credible interval of a beta distribution: alpha must be positive.
Find the credible interval of a beta distribution with alpha = 12.	NEEDS_INPUT	beta_credible_interval needs beta (Beta beta (> 0)). Example: solve('beta_credible_interval', alpha=1, beta=1, level=0.9

Credible interval for a proportion (beta-binomial) — `beta_binomial_credible_interval`

Equal-tailed credible interval of a success probability after k successes in n trials with a Beta(alpha, beta) prior.

$$p \mid k \sim \mathrm{Beta}(\alpha + k, \beta + n - k), \quad [q_{(1-c)/2},\ q_{(1+c)/2}]$$

input	meaning	condition
`alpha`	prior Beta alpha (> 0)	
`beta`	prior Beta beta (> 0)	
`successes`	number of successes k	
`trials`	number of trials n	
`level`	credibility level as a fraction (0.95 for 95%; default 0.95)	

output	meaning
`post_alpha`	posterior alpha
`post_beta`	posterior beta
`lower`	lower credible limit
`upper`	upper credible limit

- **Method:** conjugate update, then beta quantiles
- **Assumes:** alpha, $\beta > 0$; $0 \le k \le n$; 0 < level < 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('beta_binomial_credible_interval', alpha=1, beta=1, successes=0,
trials=0, level=0.9)
# -> post_alpha = 1, post_beta = 1, lower = 0.05, upper = 0.95   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('beta_binomial_credible_interval', alpha=2, beta=2, successes=7, trials=10, level=0.95)
# -> post_alpha = 9, post_beta = 5, lower = 0.385738, upper = 0.861421   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
90% credible interval for p: prior Beta(1, 1), 1 successes in 1 trials.	SOLVED	post_alpha = 2, post_beta = 1, lower = 0.223607, upper = 0.974679
With a uniform prior and 15 successes in 50 trials, find the 90% credible interval for the proportion.	SOLVED	post_alpha = 16, post_beta = 36, lower = 0.207694, upper = 0.41618
95% credible interval for p: prior Beta(2, 2), 11 successes in 10 trials.	INVALID_INPUT	Credible interval for a proportion (beta-binomial): successes cannot exceed trials.
95% credible interval for p: prior Beta(2, 2) and 7 successes.	NEEDS_INPUT	beta_binomial_credible_interval needs trials (number of trials n). Example: solve('beta_binomial_credible_interval', alp

Posterior probability that p exceeds a value — `beta_posterior_tail_probability`

P(p > c) and P(p < c) under a Beta(alpha, beta) posterior, optionally after k successes in n trials.

$$P(p > c \mid k) = 1 - I_c(\alpha + k,\ \beta + n - k)$$

input	meaning	condition
`alpha`	Beta alpha (> 0)	
`beta`	Beta beta (> 0)	
`threshold`	the value c, $0 < c < 1$	
`successes`	successes to add (default 0)	
`trials`	trials to add (default 0)	

output	meaning
`prob_above`	posterior probability P(p > c)
`prob_below`	posterior probability P(p < c)

- **Method:** regularized incomplete beta function of the (updated) posterior
- **Assumes:** alpha, $\beta > 0$; $0 < c < 1$; $0 \le k \le n$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('beta_posterior_tail_probability', alpha=1, beta=1, threshold=0.3)
# -> prob_above = 0.7, prob_below = 0.3   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('beta_posterior_tail_probability', alpha=2, beta=1, threshold=0.5)
# -> prob_above = 0.75, prob_below = 0.25   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
For a Beta(2, 1) posterior, what is the probability that p exceeds 0.5?	SOLVED	prob_above = 0.75, prob_below = 0.25
Prior Beta(1, 1) and 1 successes in 1 trials: what is the probability that p is less than 0.5?	SOLVED	prob_above = 0.75, prob_below = 0.25
For a Beta(8, 4) posterior, what is the probability that p exceeds 1.5?	INVALID_INPUT	Posterior probability that p exceeds a value: the threshold must be strictly between 0 and 1.

Jeffreys interval for a binomial proportion — `jeffreys_interval`

The equal-tailed Bayesian interval for a proportion under the Jeffreys Beta($\mathbf{1/2}$, 1/2) prior (lower limit 0 when $k = 0$ and upper limit 1 when k = n).

$$\left[B^{-1}\left(\tfrac{1-c}{2};\, k+\tfrac{1}{2},\, n-k+\tfrac{1}{2}\right),\; B^{-1}\left(\tfrac{1+c}{2};\, k+\tfrac{1}{2},\, n-k+\tfrac{1}{2}\right)\right]$$

input	meaning	condition
`successes`	number of successes k	
`trials`	number of trials n (>= 1)	
`level`	credibility level as a fraction (0.95 for 95%; default 0.95)	

output	meaning
`lower`	lower limit
`upper`	upper limit
`posterior_mean`	$(k+1/2)/(n+1)$

- **Method:** quantiles of Beta($k + \mathbf{1/2}$, n - k + 1/2)
- **Assumes:** $0 \le k \le n$; $n \ge 1$; 0 < level < 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('jeffreys_interval', successes=7, trials=50, level=0.95)
# -> lower = 0.0648717, upper = 0.255325, posterior_mean = 0.147059   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('jeffreys_interval', successes=0, trials=20, level=0.9)
# -> lower = 0, upper = 0.0904764, posterior_mean = 0.0238095   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Jeffreys interval at 95% for 7 successes in 50 trials.	SOLVED	lower = 0.0648717, upper = 0.255325, posterior_mean = 0.147059
Find the 90% Jeffreys interval with successes = 20 and trials = 20.	SOLVED	lower = 0.909524, upper = 1, posterior_mean = 0.97619
Jeffreys interval at 95% for 9 successes in 5 trials.	INVALID_INPUT	Jeffreys interval for a binomial proportion: successes cannot exceed trials.
Jeffreys interval at 95% with successes = 9.	NEEDS_INPUT	jeffreys_interval needs trials (number of trials n (>= 1)). Example: solve('jeffreys_interval', successes=7, trials=50,

Posterior predictive probability (beta-binomial) — `beta_binomial_predictive`

Probability of exactly j successes in m future trials when the success probability has a Beta(alpha, beta) posterior; the cumulative probability and the predictive mean.

$$P(J = j) = \binom{m}{j} \frac{B(\alpha + j,\ \beta + m - j)}{B(\alpha, \beta)}$$

input	meaning	condition
`alpha`	posterior alpha (> 0)	
`beta`	posterior beta (> 0)	
`future_trials`	number of future trials m (>= 1)	
`future_successes`	number of future successes j	

output	meaning
`probability`	P(exactly j successes in m trials)
`prob_at_most`	P(at most j)
`prob_more`	P(more than j)
`predictive_mean`	expected number of successes m alpha/(alpha+beta)

- **Method:** beta-binomial distribution (the binomial averaged over the beta posterior)
- **Assumes:** alpha, $\beta > 0$; $0 \le j \le m$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('beta_binomial_predictive', alpha=1, beta=1, future_trials=4,
future_successes=2)
# -> probability = 0.2, prob_at_most = 0.6, prob_more = 0.4, predictive_mean = 2   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('beta_binomial_predictive', alpha=8, beta=4, future_trials=5,
future_successes=3)
# -> probability = 0.274725, prob_at_most = 0.516484, prob_more = 0.483516, predictive_mean =
3.33333   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Posterior Beta(1, 1): probability of exactly 2 successes in the next 4 trials (posterior predictive)?	SOLVED	probability = 0.2, prob_at_most = 0.6, prob_more = 0.4, predictive_mean = 2
Posterior predictive with alpha = 2, beta = 3, future trials = 1, future successes = 1.	SOLVED	probability = 0.4, prob_at_most = 1, prob_more = 0, predictive_mean = 0.4
Posterior predictive with alpha = 2, beta = 3, future trials = 3, future successes = 4.	INVALID_INPUT	Posterior predictive probability (beta-binomial): future successes cannot exceed future trials.

Marginal likelihood (evidence) of binomial data under a beta prior — `beta_binomial_evidence`

Prior predictive probability of k successes in n trials when p ~ Beta(alpha, beta), and its logarithm.

$$p(k) = \binom{n}{k} \frac{B(\alpha + k,\ \beta + n - k)}{B(\alpha, \beta)}$$

input	meaning	condition
`alpha`	prior Beta alpha (> 0)	
`beta`	prior Beta beta (> 0)	
`successes`	number of successes k	
`trials`	number of trials n	

output	meaning
`marginal_likelihood`	P(k | n, alpha, beta)
`log_marginal_likelihood`	its natural log

- **Method:** closed form with log-beta functions
- **Assumes:** alpha, $\beta > 0$; $0 \le k \le n$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('beta_binomial_evidence', alpha=1, beta=1, successes=3, trials=12)
# -> marginal_likelihood = 0.0769231, log_marginal_likelihood = -2.56495   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('beta_binomial_evidence', alpha=2, beta=2, successes=7, trials=10)
# -> marginal_likelihood = 0.111888, log_marginal_likelihood = -2.19026   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Marginal likelihood of 7 successes in 10 trials under a Beta(1, 1) prior.	SOLVED	marginal_likelihood = 0.0909091, log_marginal_likelihood = -2.3979
Evidence for successes = 3, trials = 12 with prior alpha = 1, beta = 1.	SOLVED	marginal_likelihood = 0.0769231, log_marginal_likelihood = -2.56495
Marginal likelihood of 7 successes in 10 trials under a Beta(-1, 2) prior.	INVALID_INPUT	Marginal likelihood (evidence) of binomial data under a beta prior: alpha must be positive.

MAP estimate of a proportion (beta-binomial) — `map_beta_binomial`

The posterior mode (maximum a posteriori estimate) of p after k successes in n trials with a Beta(alpha, beta) prior, with the maximum-likelihood estimate and the posterior mean for comparison.

$$\hat{p}_{MAP} = \frac{\alpha + k - 1}{\alpha + \beta + n - 2}$$

input	meaning	condition
`alpha`	prior Beta alpha (> 0)	
`beta`	prior Beta beta (> 0)	
`successes`	number of successes k	

input	meaning	condition
`trials`	number of trials n	

output	meaning
`map`	posterior mode $(\alpha + k - 1)/(\alpha + \beta + n - 2)$
`mle`	k/n
`posterior_mean`	(alpha + k)/(alpha + beta + n)

- **Method:** maximize the log posterior (closed form of its stationary point)
- **Assumes:** $\alpha + k > 1$ and $\beta + n - k > 1$ (interior mode)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('map_beta_binomial', alpha=2, beta=2, successes=7, trials=10)
# -> map = 0.666667, mle = 0.7, posterior_mean = 0.642857   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('map_beta_binomial', alpha=3, beta=5, successes=4, trials=12)
# -> map = 0.333333, mle = 0.333333, posterior_mean = 0.35   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
MAP estimate of p with prior Beta(2, 2) and 7 successes in 10 trials.	SOLVED	map = 0.666667, mle = 0.7, posterior_mean = 0.642857
Find the posterior mode with alpha = 3, beta = 5, successes = 4, trials = 12.	SOLVED	map = 0.333333, mle = 0.333333, posterior_mean = 0.35
MAP estimate of p with prior Beta(1, 1) and 0 successes in 10 trials.	INVALID_INPUT	MAP estimate of a proportion (beta-binomial): the posterior mode lies on the boundary (alpha + k <= 1 or beta + n - k <=

Laplace (normal) approximation of a beta posterior — `laplace_approximation_beta`

Approximate a Beta(alpha, beta) posterior by a normal distribution centred at its mode with variance -1/(second derivative of the log density); the approximate credible interval.

$$\hat{p} = \frac{\alpha - 1}{\alpha + \beta - 2}, \quad \sigma^2 = \left[\frac{\alpha - 1}{\hat{p}^2} + \frac{\beta - 1}{(1 - \hat{p})^2} \right]^{-1}$$

input	meaning	condition
`alpha`	Beta alpha (> 1)	
`beta`	Beta beta (> 1)	
`level`	credibility level as a fraction (0.95 for 95%; default 0.95)	

output	meaning
`mode`	posterior mode
`laplace_sd`	Laplace standard deviation
`lower`	approximate lower limit

output	meaning
`upper`	approximate upper limit

- **Method:** second-order Taylor expansion of the log posterior at its mode
- **Assumes:** $\alpha > 1$ and $\beta > 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('laplace_approximation_beta', alpha=3, beta=3, level=0.95)
# -> mode = 0.5, laplace_sd = 0.25, lower = 0.010009, upper = 0.989991   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('laplace_approximation_beta', alpha=10, beta=4)
# -> mode = 0.75, laplace_sd = 0.125, lower = 0.505005, upper = 0.994995   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Laplace approximation of a Beta(3, 3) posterior with a 95% interval.	SOLVED	mode = 0.5, laplace_sd = 0.25, lower = 0.010009, upper = 0.989991
Use the Laplace approximation for a beta posterior with alpha = 21 and beta = 11.	SOLVED	mode = 0.666667, laplace_sd = 0.0860663, lower = 0.49798, upper = 0.835354
Laplace approximation of a Beta(1, 4) posterior.	INVALID_INPUT	Laplace (normal) approximation of a beta posterior: the Laplace approximation needs an interior mode: alpha > 1 and beta

Highest-density interval of a beta posterior — `beta_hdi`

The shortest interval holding a stated posterior probability of a unimodal Beta(alpha, beta): equal density at both ends.

$$F(u) - F(l) = c, \quad f(l) = f(u)$$

input	meaning	condition
`alpha`	Beta alpha (> 1)	
`beta`	Beta beta (> 1)	
`level`	credibility level as a fraction (0.95 for 95%; default 0.95)	

output	meaning
`lower`	lower limit
`upper`	upper limit
`width`	interval width

- **Method:** bisection on the lower limit l with $u = F^{-1}$(F(l) + level) until f(l) = f(u)
- **Assumes:** $\alpha > 1$ and $\beta > 1$ (unimodal, zero density at both ends); 0 < level < 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('beta_hdi', alpha=2, beta=2, level=0.5)
# -> lower = 0.326352, upper = 0.673648, width = 0.347296   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('beta_hdi', alpha=3, beta=9, level=0.95)
# -> lower = 0.0405552, upper = 0.483724, width = 0.443168   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
95% highest-density interval of a Beta(3, 9) posterior.	SOLVED	lower = 0.0405552, upper = 0.483724, width = 0.443168
Find the 90% HDI for a beta posterior with alpha = 1 and beta = 4.	INVALID_INPUT	Highest-density interval of a beta posterior: the highest-density interval is computed for unimodal Beta with alpha > 1
Find the 90% HDI for a beta posterior with alpha = 12.	NEEDS_INPUT	beta_hdi needs beta (Beta beta (> 1)). Example: solve('beta_hdi', alpha=2, beta=2, level=0.5).

Beta prior from a mean and standard deviation — `beta_prior_from_mean_sd`

The Beta(alpha, beta) prior with a stated mean and standard deviation (method of moments) and its effective prior sample size alpha + beta.

$$\nu = \frac{m(1-m)}{s^2} - 1, \quad \alpha = m\nu, \quad \beta = (1-m)\nu$$

input	meaning	condition
`mean`	prior mean, $0 < m < 1$	
`sd`	prior standard deviation, $sd^2 < m$(1 - m)	

output	meaning
`alpha`	alpha
`beta`	beta
`prior_sample_size`	alpha + beta

- **Method:** match the beta mean and variance
- **Assumes:** 0 < mean < 1; 0 < sd^2 < mean (1 - mean)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('beta_prior_from_mean_sd', mean=0.3, sd=0.1)
# -> alpha = 6, beta = 14, prior_sample_size = 20   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('beta_prior_from_mean_sd', mean=0.5, sd=0.2886751345948129)
# -> alpha = 1, beta = 1, prior_sample_size = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the beta prior with mean = 0.3 and standard deviation = 0.1.	SOLVED	alpha = 6, beta = 14, prior_sample_size = 20
Which beta distribution has mean 0.8 and sd 0.05?	SOLVED	alpha = 50.4, beta = 12.6, prior_sample_size = 63
Find the beta prior with mean = 0.5 and standard deviation = 0.6.	INVALID_INPUT	Beta prior from a mean and standard deviation: the variance must be smaller than mean x (1 - mean) for a beta distributi
Find the beta prior with mean = 0.5 and an unknown standard deviation.	NEEDS_INPUT	beta_prior_from_mean_sd needs sd (prior standard deviation, sd^2 < m(1 - m)). Example: solve('beta_prior_from_mean_sd',

Bayesian A/B test: probability that A beats B — `ab_test_probability_beta`

P(p_A > p_B) for independent Beta posteriors of two success probabilities (alpha_b a whole number, as after a uniform or any integer-parameter prior).

$$P(p_B > p_A) = \sum_{i=0}^{\alpha_B - 1} \frac{B(\alpha_A + i,\ \beta_A + \beta_B)}{(\beta_B + i)\, B(1 + i,\ \beta_B)\, B(\alpha_A, \beta_A)}$$

input	meaning	condition
`alpha_a`	posterior alpha of A	
`beta_a`	posterior beta of A	
`alpha_b`	posterior alpha of B (whole number)	
`beta_b`	posterior beta of B	

output	meaning
`prob_a_greater`	P(p_A > p_B)
`prob_b_greater`	P(p_B > p_A)

- **Method:** exact finite sum of beta functions (Miller's formula)
- **Assumes:** all parameters > 0; alpha_b a whole number
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('ab_test_probability_beta', alpha_a=1, beta_a=1, alpha_b=1,
beta_b=1)
# -> prob_a_greater = 0.5, prob_b_greater = 0.5   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('ab_test_probability_beta', alpha_a=2, beta_a=1, alpha_b=1,
beta_b=1)
# -> prob_a_greater = 0.666667, prob_b_greater = 0.333333   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Bayesian A/B test: alpha_a = 1, beta_a = 1, alpha_b = 1, beta_b = 1. Probability that A beats B?	SOLVED	prob_a_greater = 0.5, prob_b_greater = 0.5

question	outcome	result
With alpha_a = 2, beta_a = 1, alpha_b = 1, beta_b = 1, what is the probability that p_A is greater than p_B?	SOLVED	prob_a_greater = 0.666667, prob_b_greater = 0.333333
Bayesian A/B test: alpha_a = 21, beta_a = 81, alpha_b = 3.5, beta_b = 71. Probability that A beats B?	INVALID_INPUT	Bayesian A/B test: probability that A beats B: alpha_b must be a whole number >= 1.

Gamma-Poisson conjugate posterior of a rate — `gamma_poisson_posterior`

Update a Gamma(shape, rate) prior on a Poisson rate with observed counts: posterior parameters, mean, variance and standard deviation.

$$\lambda \mid x \sim \mathrm{Gamma}(\alpha + \textstyle\sum x_i,\ \beta + n)$$

input	meaning	condition
`shape`	prior shape alpha (> 0)	
`rate`	prior rate beta (> 0)	
`counts`	observed counts (whole numbers), one per unit interval	

output	meaning
`post_shape`	alpha + sum of counts
`post_rate`	beta + number of intervals
`posterior_mean`	posterior mean of the rate
`posterior_variance`	posterior variance
`posterior_sd`	posterior standard deviation

- **Method:** conjugate update of the gamma prior with the Poisson likelihood
- **Assumes:** shape, rate > 0; counts whole numbers >= 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('gamma_poisson_posterior', shape=2, rate=1, counts='3, 5, 4')
# -> post_shape = 14, post_rate = 4, posterior_mean = 3.5, posterior_variance = 0.875, posterior_sd
= 0.935414   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('gamma_poisson_posterior', shape=1, rate=0.5, counts=[0, 2, 1, 1])
# -> post_shape = 5, post_rate = 4.5, posterior_mean = 1.11111, posterior_variance = 0.246914,
posterior_sd = 0.496904   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Gamma-Poisson posterior: prior Gamma(2, 1), counts = [3, 5, 4]. Posterior mean of the rate?	SOLVED	post_shape = 14, post_rate = 4, posterior_mean = 3.5, posterior_variance = 0.875, posterior_sd = 0.935414
With shape = 1 and rate = 0.5 and Poisson counts = [0, 2, 1, 1], find the posterior of the rate.	SOLVED	post_shape = 5, post_rate = 4.5, posterior_mean = 1.11111, posterior_variance = 0.246914, posterior_sd = 0.496904

question	outcome	result
Gamma-Poisson posterior: prior Gamma(2, 1), counts = [3, -5, 4]. Posterior mean?	INVALID_INPUT	Gamma-Poisson conjugate posterior of a rate: Poisson counts must be whole numbers >= 0.
Gamma-Poisson posterior with shape = 2 and counts = [3, 5].	NEEDS_INPUT	gamma_poisson_posterior needs rate (prior rate beta (> 0)). Example: solve('gamma_poisson_posterior', shape=2, rate=1, c

Credible interval for a Poisson rate — `gamma_poisson_credible_interval`

Equal-tailed credible interval of a Poisson rate after observed counts with a Gamma(shape, rate) prior.

$$\left[\frac{G^{-1}_{\alpha'}\left(\frac{1-c}{2}\right)}{\beta'},\ \frac{G^{-1}_{\alpha'}\left(\frac{1+c}{2}\right)}{\beta'}\right]$$

input	meaning	condition
`shape`	prior shape (> 0)	
`rate`	prior rate (> 0)	
`counts`	observed counts	
`level`	credibility level as a fraction (0.95 for 95%; default 0.95)	

output	meaning
`post_shape`	posterior shape
`post_rate`	posterior rate
`lower`	lower limit
`upper`	upper limit

- **Method:** conjugate update, then gamma quantiles (inverse regularized incomplete gamma)
- **Assumes:** shape, rate > 0; counts whole numbers >= 0; 0 < level < 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('gamma_poisson_credible_interval', shape=1, rate=1, counts='0',
level=0.9)
# -> post_shape = 1, post_rate = 2, lower = 0.0256466, upper = 1.49787   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('gamma_poisson_credible_interval', shape=2, rate=1, counts=[3, 5,
4])
# -> post_shape = 14, post_rate = 4, lower = 1.91348, upper = 5.5576   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
90% credible interval for a Poisson rate: prior Gamma(1, 1), counts = [0].	SOLVED	post_shape = 1, post_rate = 2, lower = 0.0256466, upper = 1.49787
Find the 90% credible interval of the rate with shape = 0.5, rate = 0.001 and counts = [7].	SOLVED	post_shape = 7.5, post_rate = 1.001, lower = 3.62685, upper = 12.4854

question	outcome	result
95% credible interval for a Poisson rate: prior Gamma(2, 0), counts = [3].	INVALID_INPUT	Credible interval for a Poisson rate: the prior rate must be positive.

Equal-tailed credible interval of a gamma distribution — `gamma_credible_interval`

The central credible interval and median of a Gamma(shape, rate) posterior.

$$\left[G^{-1}\left(\tfrac{1-c}{2}\right),\ G^{-1}\left(\tfrac{1+c}{2}\right)\right]$$

input	meaning	condition
`shape`	shape (> 0)	
`rate`	rate (> 0)	
`level`	credibility level as a fraction (0.95 for 95%; default 0.95)	

output	meaning
`lower`	lower limit
`upper`	upper limit
`median`	median

- **Method:** gamma quantiles by inversion of the regularized incomplete gamma function
- **Assumes:** shape, rate > 0; 0 < level < 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('gamma_credible_interval', shape=1, rate=1, level=0.9)
# -> lower = 0.0512933, upper = 2.99573, median = 0.693147   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('gamma_credible_interval', shape=5, rate=2)
# -> lower = 0.811743, upper = 5.12079, median = 2.33545   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
90% credible interval of a Gamma(1, 2) posterior.	SOLVED	lower = 0.0256466, upper = 1.49787, median = 0.346574
Find the 90% credible interval of a gamma distribution with shape = 3 and rate = 0.5.	SOLVED	lower = 1.63538, upper = 12.5916, median = 5.34812
95% credible interval of a Gamma(5, -2) posterior.	INVALID_INPUT	Equal-tailed credible interval of a gamma distribution: the rate must be positive.

Highest-density interval of a gamma posterior — `gamma_hdi`

The shortest interval holding a stated probability of a unimodal Gamma(shape, rate): equal density at both ends.

$$G(u) - G(l) = c, \quad g(l) = g(u)$$

input	meaning	condition
`shape`	shape (> 1)	
`rate`	rate (> 0)	
`level`	credibility level as a fraction (0.95 for 95%; default 0.95)	

output	meaning
`lower`	lower limit
`upper`	upper limit
`width`	interval width

- **Method:** bisection on the lower limit with $u = G^{-1}$(G(l) + level) until the densities are equal
- **Assumes:** shape > 1; rate > 0; 0 < level < 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('gamma_hdi', shape=5, rate=2, level=0.95)
# -> lower = 0.60348, upper = 4.71511, width = 4.11163   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('gamma_hdi', shape=2, rate=1, level=0.5)
# -> lower = 0.435552, upper = 1.91795, width = 1.4824   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
95% highest-density interval of a Gamma(5, 2) posterior.	SOLVED	lower = 0.60348, upper = 4.71511, width = 4.11163
Find the 90% HDI of a gamma posterior with shape = 0.5 and rate = 0.5.	INVALID_INPUT	Highest-density interval of a gamma posterior: the highest-density interval is computed for a unimodal gamma with shape

Posterior predictive count (negative binomial) — `gamma_poisson_predictive`

Probability of exactly k events in the next unit interval when the Poisson rate has a Gamma(shape, rate) posterior; the cumulative probability and the predictive mean and variance.

$$P(X = k) = \frac{\Gamma(\alpha + k)}{\Gamma(\alpha)\, k!} \left(\frac{\beta}{\beta+1}\right)^{\alpha} \left(\frac{1}{\beta+1}\right)^{k}$$

input	meaning	condition
`shape`	posterior shape (> 0)	
`rate`	posterior rate (> 0)	
`events`	number of events k in the next interval	

output	meaning
`probability`	P(exactly k events)
`prob_at_most`	P(at most k events)
`predictive_mean`	shape/rate

output	meaning
`predictive_variance`	shape/rate (1 + 1/rate)

- **Method:** negative binomial distribution (the Poisson averaged over the gamma posterior)
- **Assumes:** shape, rate > 0; k whole number >= 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('gamma_poisson_predictive', shape=1, rate=1, events=0)
# -> probability = 0.5, prob_at_most = 0.5, predictive_mean = 1, predictive_variance = 2   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('gamma_poisson_predictive', shape=12, rate=4, events=2)
# -> probability = 0.214405, prob_at_most = 0.448051, predictive_mean = 3, predictive_variance =
3.75   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Posterior Gamma(1, 1) for a Poisson rate: probability of exactly 1 events in the next interval (posterior predictive)?	SOLVED	probability = 0.25, prob_at_most = 0.75, predictive_mean = 1, predictive_variance = 2
Posterior predictive with shape = 3, rate = 1, events = 0.	SOLVED	probability = 0.125, prob_at_most = 0.125, predictive_mean = 3, predictive_variance = 6
Posterior predictive with shape = 3, rate = 1, events = 1.5.	INVALID_INPUT	events: expected a whole number written exactly, got 1.5.

MAP estimate of a Poisson rate (gamma prior) — `map_gamma_poisson`

Posterior mode of a Poisson rate with a Gamma(shape, rate) prior and observed counts, with the MLE and the posterior mean for comparison.

$$\hat{\lambda}_{MAP} = \frac{\alpha + \sum x_i - 1}{\beta + n}$$

input	meaning	condition
`shape`	prior shape (> 0)	
`rate`	prior rate (> 0)	
`counts`	observed counts	

output	meaning
`map`	(shape + sum - 1)/(rate + n)
`mle`	sample mean of the counts
`posterior_mean`	(shape + sum)/(rate + n)

- **Method:** stationary point of the log posterior
- **Assumes:** shape + sum of counts > 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('map_gamma_poisson', shape=2, rate=1, counts='3, 5, 4')
# -> map = 3.25, mle = 4, posterior_mean = 3.5   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('map_gamma_poisson', shape=1, rate=0.5, counts=[0, 2, 1, 1])
# -> map = 0.888889, mle = 1, posterior_mean = 1.11111   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
MAP estimate of the Poisson rate with prior Gamma(2, 1) and counts = [3, 5, 4].	SOLVED	map = 3.25, mle = 4, posterior_mean = 3.5
Posterior mode of the rate with shape = 1, rate = 0.5, counts = [0, 2, 1, 1].	SOLVED	map = 0.888889, mle = 1, posterior_mean = 1.11111
MAP estimate of the Poisson rate with prior Gamma(0.5, 1) and counts = [0, 0].	INVALID_INPUT	MAP estimate of a Poisson rate (gamma prior): the posterior mode is at 0 (shape + sum of counts <= 1): no interior MAP.

Gamma-exponential conjugate posterior of a rate — `gamma_exponential_posterior`

Update a Gamma(shape, rate) prior on the rate of exponential waiting times with observed times.

$$\lambda \mid t \sim \mathrm{Gamma}(\alpha + n,\ \beta + \textstyle\sum t_i)$$

input	meaning	condition
`shape`	prior shape (> 0)	
`rate`	prior rate (> 0)	
`times`	observed waiting times (> 0)	

output	meaning
`post_shape`	shape + n
`post_rate`	rate + sum of times
`posterior_mean`	posterior mean of the rate
`posterior_variance`	posterior variance
`posterior_sd`	posterior standard deviation

- **Method:** conjugate update of the gamma prior with the exponential likelihood
- **Assumes:** shape, rate > 0; times > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('gamma_exponential_posterior', shape=2, rate=1, times='0.5, 1.2, 0.8')
# -> post_shape = 5, post_rate = 3.5, posterior_mean = 1.42857, posterior_variance = 0.408163, posterior_sd = 0.638877   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('gamma_exponential_posterior', shape=1, rate=2, times=[3, 5])
# -> post_shape = 3, post_rate = 10, posterior_mean = 0.3, posterior_variance = 0.03, posterior_sd
= 0.173205   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Gamma-exponential posterior: prior Gamma(2, 1), exponential waiting times = [0.5, 1.2, 0.8]. Posterior mean of the rate?	SOLVED	post_shape = 5, post_rate = 3.5, posterior_mean = 1.42857, posterior_variance = 0.408163, posterior_sd = 0.638877
Exponential lifetimes times = [3, 5] with prior shape = 1 and rate = 2: posterior of the rate?	SOLVED	post_shape = 3, post_rate = 10, posterior_mean = 0.3, posterior_variance = 0.03, posterior_sd = 0.173205
Exponential lifetimes times = [3, -5] with prior shape = 1 and rate = 2: posterior of the rate?	INVALID_INPUT	Gamma-exponential conjugate posterior of a rate: exponential observations must be positive.

Normal mean with known variance: posterior from data — `normal_normal_posterior`

Posterior of a normal mean theta with a N(prior mean, prior sd^2) prior, given observations with known standard deviation sigma.

$$\frac{1}{\tau_n^2} = \frac{1}{\tau_0^2} + \frac{n}{\sigma^2}, \quad \mu_n = \tau_n^2 \left(\frac{\mu_0}{\tau_0^2} + \frac{n\bar{x}}{\sigma^2} \right)$$

input	meaning	condition
`prior_mean`	prior mean mu0	
`prior_sd`	prior standard deviation tau0 (> 0)	
`sigma`	known standard deviation of each observation (> 0)	
`data`	the observations	

output	meaning
`posterior_mean`	posterior mean
`posterior_sd`	posterior standard deviation
`posterior_variance`	posterior variance

- **Method:** precision-weighted average of the prior mean and the sample mean
- **Assumes:** prior $sd > 0$; $\sigma > 0$ known
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('normal_normal_posterior', prior_mean=0, prior_sd=1, sigma=1,
data='1.2')
# -> posterior_mean = 0.6, posterior_sd = 0.707107, posterior_variance = 0.5   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('normal_normal_posterior', prior_mean=10, prior_sd=2, sigma=3,
data=[12, 9, 14])
# -> posterior_mean = 10.9524, posterior_sd = 1.30931, posterior_variance = 1.71429   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Normal-normal posterior: prior mean = 10, prior sd = 2, known sigma = 3, data = [12, 9, 14]. Posterior mean?	SOLVED	posterior_mean = 10.9524, posterior_sd = 1.30931, posterior_variance = 1.71429
Known-variance normal model with prior mean 0, prior standard deviation 1, sigma = 1 and data = [1.2]: find the posterior.	SOLVED	posterior_mean = 0.6, posterior_sd = 0.707107, posterior_variance = 0.5
Normal-normal posterior: prior mean = 10, prior sd = -2, known sigma = 3, data = [12]. Posterior?	INVALID_INPUT	Normal mean with known variance: posterior from data: the prior standard deviation must be positive.
Normal-normal posterior: prior mean = 10, known sigma = 3, data = [12]. Posterior given the prior sd?	NEEDS_INPUT	normal_normal_posterior needs prior_sd (prior standard deviation tau0 (> 0)). Example: solve('normal_normal_posterior',

Normal mean with known variance: posterior and credible interval from a sample mean — `normal_normal_summary`

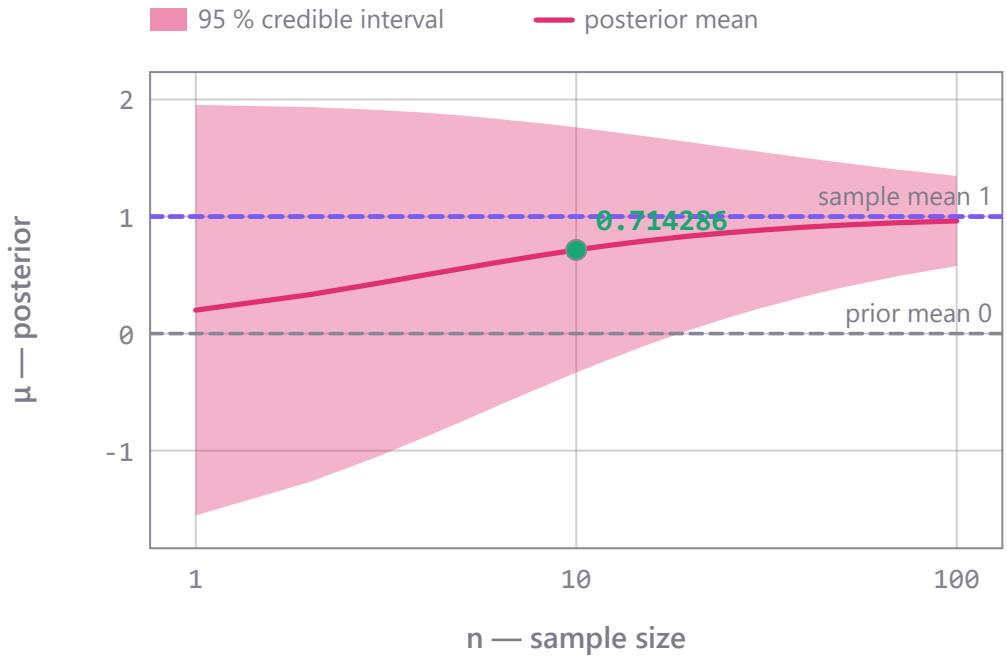
Posterior mean, standard deviation, credible interval and shrinkage factor of a normal mean from the sample mean of n observations with known sigma and a normal prior.

$$\mu_n = B\mu_0 + (1-B)\bar{x},\ B = \frac{\sigma^2/n}{\sigma^2/n + \tau_0^2}, \quad \mu_n \pm z_{(1+c)/2}\,\tau_n$$

input	meaning	condition
`prior_mean`	prior mean	
`prior_sd`	prior standard deviation (> 0)	
`sigma`	known standard deviation of one observation (> 0)	
`sample_mean`	sample mean	
`n`	sample size (>= 1)	
`level`	credibility level as a fraction (0.95 for 95%; default 0.95)	

output	meaning
`posterior_mean`	posterior mean
`posterior_sd`	posterior standard deviation
`lower`	lower credible limit
`upper`	upper credible limit
`shrinkage`	weight on the prior mean $(\sigma^2/n)/(\sigma^2/n + tau0^2)$

- **Method:** normal-normal conjugate update, interval from normal quantiles
- **Assumes:** prior $sd > 0$; $\sigma > 0$; $n \geq 1$; 0 < level < 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A normal prior (mean 0, sd 1) and data with σ = 2 and sample mean 1: the package's posterior mean and 95 % credible interval for 17 sample sizes from 1 to 100, moving from the prior towards the data; marked: n = 10, posterior mean = 0.714286, interval [-0.333359, 1.76193].

```
bayesian_statistics_ai999x.solve('normal_normal_summary', prior_mean=0, prior_sd=1, sigma=2, sample_mean=1, n=4)
# -> posterior_mean = 0.5, posterior_sd = 0.707107, lower = -0.885904, upper = 1.8859, shrinkage = 0.5   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('normal_normal_summary', prior_mean=100, prior_sd=15, sigma=20, sample_mean=110, n=25, level=0.95)
# -> posterior_mean = 109.336, posterior_sd = 3.86494, lower = 101.761, upper = 116.911, shrinkage = 0.06639   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Normal prior with prior mean = 0, prior sd = 1; sample mean = 1 from n = 4 with known sigma = 2. Posterior mean and 95% credible interval?	SOLVED	posterior_mean = 0.5, posterior_sd = 0.707107, lower = -0.885904, upper = 1.8859, shrinkage = 0.5
Prior mean 0, prior standard deviation 2, known sigma = 4, sample mean 1.5, sample size 16: posterior?	SOLVED	posterior_mean = 1.2, posterior_sd = 0.894427, lower = -0.553045, upper = 2.95305, shrinkage = 0.2
Prior mean 0, prior standard deviation 2, known sigma = 4, sample mean 1.5, sample size 0: posterior?	INVALID_INPUT	n: must be at least 1, got 0.

Posterior predictive distribution of a new observation — `normal_posterior_predictive`

Mean, standard deviation and tail probabilities of the next observation in the normal model with known sigma, after a sample mean of n observations and a normal prior.

$$Y_{new} \mid \bar{x} \sim N\left(\mu_n,\ \tau_n^2 + \sigma^2\right)$$

input	meaning	condition
`prior_mean`	prior mean	
`prior_sd`	prior standard deviation (> 0)	
`sigma`	known standard deviation (> 0)	
`sample_mean`	sample mean	
`n`	sample size (>= 1)	
`threshold`	value c for P(Y > c)	

output	meaning
`predictive_mean`	posterior mean
`predictive_sd`	sqrt(posterior variance + sigma^2)
`prob_above`	P(Y_new > c)
`prob_below`	P(Y_new < c)

- **Method:** predictive variance = posterior variance + sampling variance
- **Assumes:** prior $sd > 0$; $\sigma > 0$; $n \geq 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('normal_posterior_predictive', prior_mean=0, prior_sd=1, sigma=2,
sample_mean=1, n=4, threshold=0.5)
# -> predictive_mean = 0.5, predictive_sd = 2.12132, prob_above = 0.5, prob_below = 0.5   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('normal_posterior_predictive', prior_mean=10, prior_sd=3, sigma=2,
sample_mean=12, n=5, threshold=15)
# -> predictive_mean = 11.8367, predictive_sd = 2.17594, prob_above = 0.0730078, prob_below =
0.926992   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Posterior predictive: prior mean = 0, prior sd = 1, known sigma = 2, sample mean = 1, n = 4. Probability that the next observation exceeds 0.5?	SOLVED	predictive_mean = 0.5, predictive_sd = 2.12132, prob_above = 0.5, prob_below = 0.5
Predictive distribution of a new observation with prior mean 10, prior sd 3, sigma = 0, sample mean 12, sample size 5 and threshold = 15.	INVALID_INPUT	Posterior predictive distribution of a new observation: sigma must be positive.

Normal model with unknown mean and variance (normal-inverse-gamma) — `normal_inverse_gamma_posterior`

Conjugate normal-inverse-gamma update for normal data with unknown mean and variance: posterior parameters, posterior mean of the variance and the Student-t credible interval for the mean.

$$\beta_n = \beta_0 + \tfrac{1}{2}\sum (x_i - \bar{x})^2 + \frac{\kappa_0 n (\bar{x} - \mu_0)^2}{2(\kappa_0 + n)}, \quad \mu \mid x \sim t_{2\alpha_n}\left(\mu_n,\ \frac{\beta_n}{\alpha_n \kappa_n}\right)$$

input	meaning	condition
`mu0`	prior location of the mean	
`kappa0`	prior pseudo-observations for the mean (> 0)	
`alpha0`	prior inverse-gamma shape (> 0)	
`beta0`	prior inverse-gamma scale (> 0)	
`data`	the observations	
`level`	credibility level as a fraction (0.95 for 95%; default 0.95)	

output	meaning
`mu_n`	posterior location
`kappa_n`	$kappa0 + n$
`alpha_n`	$alpha0 + n/2$
`beta_n`	posterior scale
`sigma2_mean`	posterior mean of σ^2 $(alpha_n > 1)$
`lower`	lower credible limit for the mean
`upper`	upper credible limit for the mean

- **Method:** conjugate update; marginal posterior of the mean is Student t with 2 alpha_n degrees of freedom
- **Assumes:** kappa0, alpha0, $beta0 > 0$; 0 < level < 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('normal_inverse_gamma_posterior', mu0=0, kappa0=1, alpha0=2,
beta0=2, data='1, -1')
# -> mu_n = 0, kappa_n = 3, alpha_n = 3, beta_n = 3, sigma2_mean = 1.5, lower = -1.41273, upper =
1.41273   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('normal_inverse_gamma_posterior', mu0=10, kappa0=0.5, alpha0=3,
beta0=4, data=[9, 11, 12])
# -> mu_n = 10.5714, kappa_n = 3.5, alpha_n = 4.5, beta_n = 6.42857, sigma2_mean = 1.83673, lower =
9.12619, upper = 12.0167   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Normal-inverse-gamma posterior: mu0 = 0, kappa0 = 1, alpha0 = 2, beta0 = 2, data = [1, -1]. 95% credible interval for the mean?	SOLVED	mu_n = 0, kappa_n = 3, alpha_n = 3, beta_n = 3, sigma2_mean = 1.5, lower = -1.41273, upper = 1.41273
Unknown mean and variance with mu0 = 10, kappa0 = 0, alpha0 = 3, beta0 = 4 and data = [9, 11, 12].	INVALID_INPUT	Normal model with unknown mean and variance (normal-inverse-gamma): kappa0 must be positive.

Dirichlet-multinomial conjugate posterior — `dirichlet_multinomial_posterior`

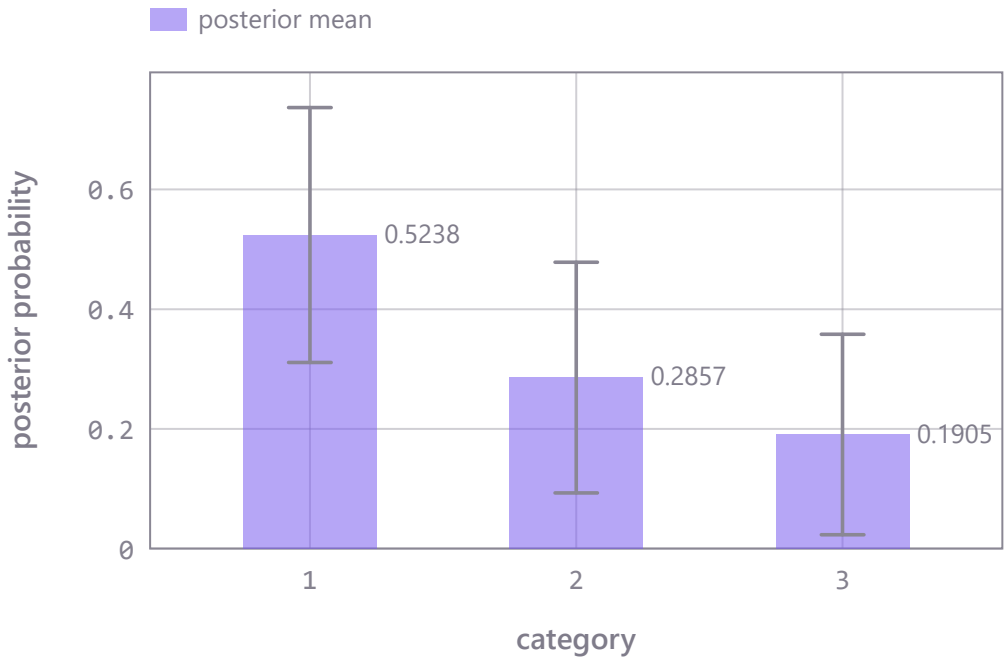
Update a Dirichlet(alpha_1, ..., alpha_k) prior on category probabilities with category counts: posterior concentrations, means and variances.

$$\theta \mid n \sim \mathrm{Dir}(\alpha_1 + n_1, \ldots, \alpha_k + n_k), \quad E[\theta_j] = \frac{\alpha_j + n_j}{\alpha_0 + n}$$

input	meaning	condition
`alpha`	prior concentrations alpha_j (> 0)	
`counts`	observed category counts	

output	meaning
`post_alpha`	alpha_j + n_j
`posterior_means`	posterior means $(alpha_j + n_j)/(alpha_0 + n)$
`posterior_variances`	posterior variances

- **Method:** conjugate update of the Dirichlet prior with the multinomial likelihood
- **Assumes:** $alpha_j > 0$; counts whole numbers >= 0; same length
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A Dirichlet(1, 1, 1) prior over 3 categories updated with the counts 10, 5, 3: the package's posterior Dirichlet(11, 6, 4), posterior means 0.52381, 0.285714, 0.190476; whiskers ±2 posterior standard deviations (0.106479, 0.0963143, 0.083719).

```
bayesian_statistics_ai999x.solve('dirichlet_multinomial_posterior', alpha='1, 1, 1', counts='10, 5, 3')
# -> post_alpha = [11.0, 6.0, 4.0], posterior_means = [0.52380952381, 0.285714285714, 0.190476190476], posterior_variances = [0.0113378684807, 0.00927643784787, 0.00700886415172]  (SOLVED)
```

```
bayesian_statistics_ai999x.solve('dirichlet_multinomial_posterior', alpha=[2, 2, 2, 2], counts=[3, 0, 7, 1])
# -> post_alpha = [5.0, 2.0, 9.0, 3.0], posterior_means = [0.263157894737, 0.105263157895, 0.473684210526, 0.157894736842], posterior_variances = [0.00969529085873, 0.00470914127424, 0.0124653739612, 0.00664819944598]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Dirichlet-multinomial posterior with alpha = [1, 1, 1] and counts = [10, 5, 3]: posterior means?	SOLVED	post_alpha = [11.0, 6.0, 4.0], posterior_means = [0.52380952381, 0.285714285714, 0.190476190476], posterior_variances = [0.0113378684807, 0.00927643784787, 0.00700886415172]
Dirichlet prior concentrations [2, 2, 2, 2] updated with counts [3, 0, 7, 1].	SOLVED	post_alpha = [5.0, 2.0, 9.0, 3.0], posterior_means = [0.263157894737, 0.105263157895, 0.473684210526, 0.157894736842], posterior_variances = [0.00969529085873, 0.00470914127424, 0.0124653739612, 0.00664819944598]
Dirichlet-multinomial posterior with alpha = [1, 1, 1] and counts = [10, 5].	INVALID_INPUT	Dirichlet-multinomial conjugate posterior: the counts must have 3 values (one per entry of the other list).

Dirichlet-multinomial probability of a count vector (evidence) —

`dirichlet_multinomial_evidence`

Prior predictive probability (marginal likelihood) of observed category counts when the category probabilities have a Dirichlet prior.

$$P(n \mid \alpha) = \frac{n!\,\Gamma(\alpha_0)}{\Gamma(n+\alpha_0)} \prod_j \frac{\Gamma(n_j+\alpha_j)}{n_j!\,\Gamma(\alpha_j)}$$

input	meaning	condition
`alpha`	prior concentrations (> 0)	
`counts`	category counts	

output	meaning
`probability`	P(counts | alpha)
`log_probability`	its natural log

- **Method:** closed form with log-gamma functions
- **Assumes:** $alpha_j > 0$; counts whole numbers >= 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('dirichlet_multinomial_evidence', alpha='1, 1, 1', counts='2, 1, 0')
# -> probability = 0.1, log_probability = -2.30259   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('dirichlet_multinomial_evidence', alpha=[1, 1], counts=[3, 3])
# -> probability = 0.142857, log_probability = -1.94591   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Dirichlet-multinomial evidence for counts = [2, 1, 0] with alpha = [1, 1, 1].	SOLVED	probability = 0.1, log_probability = -2.30259

question	outcome	result
Marginal likelihood of counts [3, 3] under a Dirichlet prior with alpha = [1, 1].	SOLVED	probability = 0.142857, log_probability = -1.94591
Dirichlet-multinomial evidence for counts = [2, 1, 0] with alpha = [1, -1, 1].	INVALID_INPUT	Dirichlet-multinomial probability of a count vector (evidence): Dirichlet concentrations must be positive.

Bayes factor for two simple binomial hypotheses — `bayes_factor_binomial_simple`

Bayes factor BF10 for H1: $p = p_1$ against H0: $p = p_0$ after k successes in n trials, its log10 and the posterior probability of H1 with equal prior odds.

$$BF_{10} = \left(\frac{p_1}{p_0}\right)^{k} \left(\frac{1-p_1}{1-p_0}\right)^{n-k}$$

input	meaning	condition
`successes`	successes k	
`trials`	trials n (>= 1)	
`p0`	success probability under H0	
`p1`	success probability under H1	

output	meaning
`bf10`	Bayes factor H1 : H0
`log10_bf10`	log10 of BF10
`posterior_h1`	P(H1 | data) with prior odds 1

- **Method:** likelihood ratio of the two simple hypotheses
- **Assumes:** $0 < p_0, p_1 < 1; 0 \le k \le n$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('bayes_factor_binomial_simple', successes=1, trials=1, p0=0.5,
p1=0.8)
# -> bf10 = 1.6, log10_bf10 = 0.20412, posterior_h1 = 0.615385   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('bayes_factor_binomial_simple', successes=14, trials=20, p0=0.5,
p1=0.7)
# -> bf10 = 5.18442, log10_bf10 = 0.7147, posterior_h1 = 0.838303   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Bayes factor for p1 = 0.7 against p0 = 0.5 after 14 successes in 20 trials.	SOLVED	bf10 = 5.18442, log10_bf10 = 0.7147, posterior_h1 = 0.838303
Compute the Bayes factor with successes = 1, trials = 1, p0 = 0.5, p1 = 0.8.	SOLVED	bf10 = 1.6, log10_bf10 = 0.20412, posterior_h1 = 0.615385
Compute the Bayes factor with successes = 3, trials = 10, p0 = 1.5, p1 = 0.2.	INVALID_INPUT	Bayes factor for two simple binomial hypotheses: p0 must be strictly between 0 and 1.

Bayes factor for a point null on a proportion (beta prior under H1) —

`bayes_factor_point_null_binomial`

BF10 for H1: p ~ Beta(alpha, beta) against H0: $p = p_0$ after k successes in n trials (default uniform prior), with BF01 and the posterior probability of H1 at equal prior odds.

$$BF_{10} = \frac{B(\alpha + k,\ \beta + n - k)/B(\alpha, \beta)}{p_0^k (1 - p_0)^{n-k}}$$

input	meaning	condition
`successes`	successes k	
`trials`	trials n (>= 1)	
`p0`	null value p0	
`alpha`	Beta alpha under H1 (default 1)	
`beta`	Beta beta under H1 (default 1)	

output	meaning
`bf10`	Bayes factor H1 : H0
`bf01`	$1/BF10$
`log10_bf10`	log10 BF10
`posterior_h1`	P(H1 | data) with prior odds 1

- **Method:** ratio of the beta-binomial marginal likelihood to the binomial likelihood at p0
- **Assumes:** $0 < p_0 < 1$; alpha, $\beta > 0$; $0 \le k \le n$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('bayes_factor_point_null_binomial', successes=1, trials=1, p0=0.5)
# -> bf10 = 1, bf01 = 1, log10_bf10 = 1.44649e-16, posterior_h1 = 0.5   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('bayes_factor_point_null_binomial', successes=2, trials=2, p0=0.5)
# -> bf10 = 1.33333, bf01 = 0.75, log10_bf10 = 0.124939, posterior_h1 = 0.571429   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Bayes factor against the point null p0 = 0.5 with a uniform prior: 2 successes in 2 trials.	SOLVED	bf10 = 1.33333, bf01 = 0.75, log10_bf10 = 0.124939, posterior_h1 = 0.571429
Compute the Bayes factor for p0 = 0.25 with successes = 9, trials = 20, alpha = 2, beta = 2.	SOLVED	bf10 = 2.50393, bf01 = 0.399372, log10_bf10 = 0.398623, posterior_h1 = 0.714606
Compute the Bayes factor for p0 = 0 with successes = 9, trials = 20.	INVALID_INPUT	Bayes factor for a point null on a proportion (beta prior under H1): p0 must be strictly between 0 and 1.

Bayes factor for a normal mean (known variance) — `bayes_factor_normal_mean`

BF10 for H1: mu ~ N(mu0, tau^2) against H0: $mu = mu0$, from the sample mean of n observations with known sigma; BF01 and the posterior probability of H1 at equal prior odds.

$$BF_{01} = \frac{p(\mu_0 \mid \bar{x}, H_1)}{p(\mu_0 \mid H_1)}$$

input	meaning	condition
`sample_mean`	sample mean	
`n`	sample size (>= 1)	
`sigma`	known standard deviation (> 0)	
`mu0`	null value mu0	
`prior_sd`	prior standard deviation tau of mu under H1 (> 0)	

output	meaning
`bf10`	Bayes factor H1 : H0
`bf01`	$1/BF10$
`log10_bf10`	log10 BF10
`posterior_h1`	P(H1 | data) with prior odds 1

- **Method:** Savage-Dickey density ratio of the posterior and prior at mu0
- **Assumes:** sigma, $\tau > 0$; $n \geq 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('bayes_factor_normal_mean', sample_mean=0, n=1, sigma=1, mu0=0,
prior_sd=1)
# -> bf10 = 0.707107, bf01 = 1.41421, log10_bf10 = -0.150515, posterior_h1 = 0.414214   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('bayes_factor_normal_mean', sample_mean=0.5, n=25, sigma=2, mu0=0,
prior_sd=1)
# -> bf10 = 0.728324, bf01 = 1.37302, log10_bf10 = -0.137675, posterior_h1 = 0.421405   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Bayes factor for a normal mean: sample mean = 0, n = 1, sigma = 1, mu0 = 0, prior sd = 1.	SOLVED	bf10 = 0.707107, bf01 = 1.41421, log10_bf10 = -0.150515, posterior_h1 = 0.414214
Compute the Bayes factor with xbar = 101.2, n = 50, known sigma = 10, mu0 = 100 and tau = 5.	SOLVED	bf10 = 0.379838, bf01 = 2.6327, log10_bf10 = -0.420402, posterior_h1 = 0.275277
Compute the Bayes factor with xbar = 101.2, n = 50, known sigma = -10, mu0 = 100 and tau = 5.	INVALID_INPUT	Bayes factor for a normal mean (known variance): sigma must be positive.

Posterior model probabilities from log evidences — `posterior_model_probabilities`

Posterior probabilities of several models from their log marginal likelihoods and prior model probabilities (default equal), computed stably in log space.

$$P(M_i \mid D) = \frac{P(D \mid M_i)P(M_i)}{\sum_j P(D \mid M_j)P(M_j)}$$

input	meaning	condition
`log_evidences`	natural-log marginal likelihoods of the models	
`priors`	prior model probabilities (default equal)	

output	meaning
`posterior`	posterior model probabilities
`best_model`	number of the most probable model
`best_probability`	its posterior probability

- **Method:** log-sum-exp normalization of log prior + log evidence
- **Assumes:** prior probabilities sum to 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('posterior_model_probabilities', log_evidences='-1, -1')
# -> posterior = [0.5, 0.5], best_model = 1, best_probability = 0.5   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('posterior_model_probabilities', log_evidences=[0,
-1.0986122886681098], priors=[0.5, 0.5])
# -> posterior = [0.75, 0.25], best_model = 1, best_probability = 0.75   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Posterior model probabilities from log marginal likelihoods = [-10, -10, -10].	SOLVED	posterior = [0.333333333333, 0.333333333333, 0.333333333333], best_model = 1, best_probability = 0.333333
Models with log evidences [-100, -101] and prior model probabilities [0.3, 0.7]: posterior probabilities?	SOLVED	posterior = [0.538101526224, 0.461898473776], best_model = 1, best_probability = 0.538102
Models with log evidences [-100, -101] and prior model probabilities [0.3, 0.8]: posterior?	INVALID_INPUT	Posterior model probabilities from log evidences: the prior model probabilities must sum to 1 (they sum to 1.1).

Best action by expected monetary value — `expected_value_decision`

Expected payoff of each action over uncertain states with known probabilities, and the action with the highest expected value.

$$EV(a) = \sum_s P(s)\,u(a,s), \quad a^* = \arg\max_a EV(a)$$

input	meaning	condition
`payoffs`	payoff matrix: one row per action, one column per state	
`probabilities`	state probabilities (sum to 1)	

output	meaning
`expected_values`	expected payoff of each action
`best_action`	number (1-based) of the best action (first if tied)
`best_expected_value`	its expected payoff

- **Method:** expected value of each row
- **Assumes:** probabilities sum to 1; one probability per column
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('expected_value_decision', payoffs='[[100, -20], [40, 30]]',
probabilities='0.4, 0.6')
# -> expected_values = [28.0, 34.0], best_action = 2, best_expected_value = 34   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('expected_value_decision', payoffs=[[5, 1, 0], [3, 3, 3]],
probabilities=[0.2, 0.5, 0.3])
# -> expected_values = [1.5, 3.0], best_action = 2, best_expected_value = 3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Expected value decision: payoffs = [[100, -20], [40, 30]], probabilities = [0.4, 0.6]. Best action?	SOLVED	expected_values = [28.0, 34.0], best_action = 2, best_expected_value = 34
Which action maximizes expected monetary value with payoff matrix [[5, 1, 0], [3, 3, 3]] and state probabilities [0.2, 0.5, 0.3]?	SOLVED	expected_values = [1.5, 3.0], best_action = 2, best_expected_value = 3
Expected value decision: payoffs = [[100, -20], [40, 30]], probabilities = [0.4, 0.4].	INVALID_INPUT	Best action by expected monetary value: the state probabilities must sum to 1 (they sum to 0.8).
Expected value decision: payoffs = [[100, -20], [40, 30]]. Best action?	NEEDS_INPUT	expected_value_decision needs probabilities (state probabilities (sum to 1)). Example: solve('expected_value_decision',

Decision under uncertainty: maximin, maximax, Laplace, minimax regret, Hurwicz — `decision_criteria_under_uncertainty`

The best action and its criterion value when state probabilities are unknown, by a stated criterion.

$$\text{maximin: } \max_a \min_s u, \quad \text{regret } r(a,s) = \max_{a'} u(a',s) - u(a,s)$$

input	meaning	condition
`payoffs`	payoff matrix: one row per action, one column per state	
`criterion`	maximin, maximax, laplace, minimax regret or hurwicz	
`optimism`	Hurwicz optimism coefficient alpha (Hurwicz only)	

output	meaning
`scores`	criterion score of each action (maximum regret for minimax regret)
`best_action`	number of the chosen action (first if tied)
`criterion_value`	criterion value of the chosen action

- **Method:** row minima / maxima / means / Hurwicz mixtures, or maximum regret per row
- **Assumes:** $0 \leq \alpha \leq 1$ for Hurwicz
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('decision_criteria_under_uncertainty', payoffs='[[100, -20], [40,
30], [60, 10]]', criterion='maximin')
# -> scores = [-20.0, 30.0, 10.0], best_action = 2, criterion_value = 30   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('decision_criteria_under_uncertainty', payoffs=[[5, 1, 0], [3, 3,
3]], criterion='minimax regret')
# -> scores = [3.0, 2.0], best_action = 2, criterion_value = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Use the maximin criterion with payoffs = [[100, -20], [40, 30], [60, 10]].	SOLVED	scores = [-20.0, 30.0, 10.0], best_action = 2, criterion_value = 30
Minimax regret decision for payoff matrix [[5, 1, 0], [3, 3, 3]].	SOLVED	scores = [3.0, 2.0], best_action = 2, criterion_value = 2
Hurwicz criterion with optimism = 0.6 and payoffs = [[100, -20], [40, 30]].	SOLVED	scores = [52.0, 36.0], best_action = 1, criterion_value = 52
Hurwicz criterion with payoffs = [[100, -20], [40, 30]].	INVALID_INPUT	Decision under uncertainty: maximin, maximax, Laplace, minimax regret, Hurwicz: the Hurwicz criterion needs the optimism

Expected value of perfect information (EVPI) — `expected_value_of_perfect_information`

The most a decision maker should pay to learn the true state before acting: expected value with perfect information minus the best expected value without it.

$$EVPI = \sum_{s} P(s) \max_{a} u(a, s) - \max_{a} \sum_{s} P(s)\, u(a, s)$$

input	meaning	condition
`payoffs`	payoff matrix: one row per action, one column per state	
`probabilities`	state probabilities (sum to 1)	

output	meaning
`evpi`	expected value of perfect information
`ev_with_perfect_information`	expected value of the best action in each state
`best_expected_value`	best expected value without information

- **Method:** E[max over actions] - max over actions of E
- **Assumes:** probabilities sum to 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('expected_value_of_perfect_information', payoffs='[[100, -20], [40, 30]]', probabilities='0.4, 0.6')
# -> evpi = 24, ev_with_perfect_information = 58, best_expected_value = 34   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('expected_value_of_perfect_information', payoffs=[[5, 1, 0], [3, 3, 3]], probabilities=[0.2, 0.5, 0.3])
# -> evpi = 0.4, ev_with_perfect_information = 3.4, best_expected_value = 3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Expected value of perfect information: payoffs = [[100, -20], [40, 30]], probabilities = [0.4, 0.6].	SOLVED	evpi = 24, ev_with_perfect_information = 58, best_expected_value = 34
Find the EVPI for payoff matrix [[5, 1, 0], [3, 3, 3]] and state probabilities [0.2, 0.5, 0.3].	SOLVED	evpi = 0.4, ev_with_perfect_information = 3.4, best_expected_value = 3
Find the EVPI for payoff matrix [[5, 1, 0], [3, 3, 3]] and state probabilities [0.2, 0.8].	INVALID_INPUT	Expected value of perfect information (EVPI): the state probabilities must have 3 values (one per entry of the other lis

Expected value of sample information (EVSI) — `expected_value_of_sample_information`

Value of an imperfect test before acting: for each possible signal, update the state probabilities by Bayes' theorem, choose the best action, and compare the expected value with that of acting now.

$$EVSI = \sum_{z} P(z) \max_{a} \sum_{s} P(s \mid z)\, u(a, s) - \max_{a} \sum_{s} P(s)\, u(a, s)$$

input	meaning	condition
`payoffs`	payoff matrix: one row per action, one column per state	
`priors`	prior state probabilities (sum to 1)	
`likelihoods`	likelihood matrix P(signal | state): one row per signal, one column per state	

output	meaning
`evsi`	expected value of sample information
`ev_with_information`	expected value with the test
`ev_without_information`	best expected value without the test
`signal_probabilities`	probability of each signal

output	meaning
`best_action_per_signal`	best action after each signal

- **Method:** preposterior analysis: Bayes' theorem per signal, best action per posterior, weighted by P(signal)
- **Assumes:** priors sum to 1; each likelihood column sums to 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('expected_value_of_sample_information', payoffs='[[100, -20], [40, 30]]', priors='0.4, 0.6', likelihoods='[[0.8, 0.3], [0.2, 0.7]]')
# -> evsi = 10.2, ev_with_information = 44.2, ev_without_information = 34, signal_probabilities = [0.5, 0.5], best_action_per_signal = [1, 2]   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('expected_value_of_sample_information', payoffs=[[50, -10], [0, 0]], priors=[0.5, 0.5], likelihoods=[[0.9, 0.2], [0.1, 0.8]])
# -> evsi = 1.5, ev_with_information = 21.5, ev_without_information = 20, signal_probabilities = [0.55, 0.45], best_action_per_signal = [1, 2]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the EVSI with payoff matrix [[50, -10], [0, 0]], prior probabilities [0.5, 0.5] and likelihood matrix [[0.9, 0.2], [0.1, 0.8]].	SOLVED	evsi = 1.5, ev_with_information = 21.5, ev_without_information = 20, signal_probabilities = [0.55, 0.45], best_action_per_signal = [1, 2]
Expected value of sample information: payoffs = [[100, -20], [40, 30]], priors = [0.4, 0.6], likelihoods = [[0.8, 0.3], [0.3, 0.7]].	INVALID_INPUT	Expected value of sample information (EVSI): each column of the likelihood matrix (P(signal | state)) must sum to 1.

Bayes action: minimum posterior expected loss — `bayes_action_minimum_expected_loss`

Posterior expected loss of each action from a loss matrix and (posterior) state probabilities; the Bayes action minimizes it.

$$\rho(a) = \sum_{s} P(s \mid x)\, L(a, s), \quad a^* = \arg\min_{a} \rho(a)$$

input	meaning	condition
`losses`	loss matrix: one row per action, one column per state	
`probabilities`	posterior state probabilities (sum to 1)	

output	meaning
`expected_losses`	expected loss of each action
`bayes_action`	action with least expected loss
`bayes_risk`	its expected loss

- **Method:** expected loss per row, minimum
- **Assumes:** probabilities sum to 1

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('bayes_action_minimum_expected_loss', losses='[[0, 10], [5, 0]]',
probabilities='0.3, 0.7')
# -> expected_losses = [7.0, 1.5], bayes_action = 2, bayes_risk = 1.5   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('bayes_action_minimum_expected_loss', losses=[[0, 4, 8], [2, 2,
2]], probabilities=[0.5, 0.3, 0.2])
# -> expected_losses = [2.8, 2.0], bayes_action = 2, bayes_risk = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Bayes action: losses = [[0, 10], [5, 0]], probabilities = [0.3, 0.7]. Minimum expected loss?	SOLVED	expected_losses = [7.0, 1.5], bayes_action = 2, bayes_risk = 1.5
Which action minimizes the expected loss with loss matrix [[0, 4, 8], [2, 2, 2]] and probabilities [0.5, 0.3, 0.2]?	SOLVED	expected_losses = [2.8, 2.0], bayes_action = 2, bayes_risk = 2
Bayes action: losses = [[0, 10], [5, 0]], probabilities = [0.3, 0.9].	INVALID_INPUT	Bayes action: minimum posterior expected loss: the state probabilities must sum to 1 (they sum to 1.2).

Expected utility and certainty equivalent (exponential utility) — `certainty_equivalent_exponential_utility`

Expected utility, certainty equivalent and risk premium of a lottery for a risk-averse decision maker with exponential utility u(x) = 1 - exp(-x/R).

$$CE = -R \ln \sum_i p_i e^{-x_i/R}$$

input	meaning	condition
`outcomes`	monetary outcomes	
`probabilities`	their probabilities (sum to 1)	
`risk_tolerance`	risk tolerance R (> 0)	

output	meaning
`expected_utility`	E[$1 - exp$(-x/R)]
`certainty_equivalent`	-R ln E[exp(-x/R)]
`risk_premium`	expected value - certainty equivalent
`expected_value`	expected value

- **Method:** log-sum-exp evaluation of E[exp(-x/R)]
- **Assumes:** probabilities sum to 1; $R > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('certainty_equivalent_exponential_utility', outcomes='1000, 0',
probabilities='0.5, 0.5', risk_tolerance=500)
# -> expected_utility = 0.432332, certainty_equivalent = 283.11, risk_premium = 216.89,
expected_value = 500   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('certainty_equivalent_exponential_utility', outcomes=[200, 50,
-100], probabilities=[0.3, 0.5, 0.2], risk_tolerance=250)
# -> expected_utility = 0.157471, certainty_equivalent = 42.8368, risk_premium = 22.1632,
expected_value = 65   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Exponential utility with risk tolerance = 500: outcomes = [1000, 0], probabilities = [0.5, 0.5]. Certainty equivalent?	SOLVED	expected_utility = 0.432332, certainty_equivalent = 283.11, risk_premium = 216.89, expected_value = 500
Exponential utility with risk tolerance = -5: outcomes = [1000, 0], probabilities = [0.5, 0.5].	INVALID_INPUT	Expected utility and certainty equivalent (exponential utility): the risk tolerance must be positive.

Certainty equivalent with logarithmic utility — `certainty_equivalent_log_utility`

Expected log utility, certainty equivalent (the weighted geometric mean) and risk premium of a lottery over positive wealth levels.

$$CE = \exp\left(\sum_i p_i \ln x_i\right) = \prod_i x_i^{p_i}$$

input	meaning	condition
`outcomes`	positive wealth levels	
`probabilities`	their probabilities (sum to 1)	

output	meaning
`expected_log_utility`	E[ln x]
`certainty_equivalent`	exp(E[ln x])
`risk_premium`	E[x] - CE
`expected_value`	E[x]

- **Method:** weighted geometric mean
- **Assumes:** outcomes > 0; probabilities sum to 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('certainty_equivalent_log_utility', outcomes='100, 400',
probabilities='0.5, 0.5')
# -> expected_log_utility = 5.29832, certainty_equivalent = 200, risk_premium = 50, expected_value
= 250   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('certainty_equivalent_log_utility', outcomes=[50, 100, 200],
probabilities=[0.25, 0.5, 0.25])
# -> expected_log_utility = 4.60517, certainty_equivalent = 100, risk_premium = 12.5,
expected_value = 112.5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Logarithmic utility: outcomes = [100, 400], probabilities = [0.5, 0.5]. Certainty equivalent?	SOLVED	expected_log_utility = 5.29832, certainty_equivalent = 200, risk_premium = 50, expected_value = 250
Find the certainty equivalent under log utility of wealth levels [50, 100, 200] with probabilities [0.25, 0.5, 0.25].	SOLVED	expected_log_utility = 4.60517, certainty_equivalent = 100, risk_premium = 12.5, expected_value = 112.5
Logarithmic utility: outcomes = [-100, 400], probabilities = [0.5, 0.5].	INVALID_INPUT	Certainty equivalent with logarithmic utility: logarithmic utility needs positive outcomes (wealth levels).

Break-even (indifference) probability of two actions — `break_even_probability`

The probability of state 1 at which two actions have equal expected value, and which action is preferred above it.

$$p^* = \frac{u_{22} - u_{12}}{(u_{11} - u_{12}) - (u_{21} - u_{22})}$$

input	meaning	condition
`payoffs`	2 x 2 payoff matrix [[action 1 in state 1, in state 2], [action 2 ...]]	

output	meaning
`break_even`	probability of state 1 at which the expected values are equal
`preferred_above`	action preferred when P(state 1) exceeds it
`expected_value_at_break_even`	the common expected value there

- **Method:** intersection of the two expected-value lines
- **Assumes:** a crossing strictly between 0 and 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('break_even_probability', payoffs='[[100, -20], [40, 30]]')
# -> break_even = 0.454545, preferred_above = 1, expected_value_at_break_even = 34.5455   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('break_even_probability', payoffs=[[50, 0], [20, 10]])
# -> break_even = 0.25, preferred_above = 1, expected_value_at_break_even = 12.5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Break-even probability for payoffs = [[100, -20], [40, 30]].	SOLVED	break_even = 0.454545, preferred_above = 1, expected_value_at_break_even = 34.5455
Break-even probability for payoffs = [[100, 50], [40, 30]].	INVALID_INPUT	Break-even (indifference) probability of two actions: one action is at least as good for every probability: there is no

Decision threshold probability (treat / do not treat) — `treatment_threshold`

The probability above which acting (treating) has lower expected loss than not acting, from the loss of a false positive and of a false negative; the decision for a given probability.

$$p^* = \frac{L_{FP}}{L_{FP} + L_{FN}}$$

input	meaning	condition
`loss_fp`	loss of acting when the condition is absent (false positive)	
`loss_fn`	loss of not acting when the condition is present (false negative)	
`probability`	probability of the condition (optional)	

output	meaning
`threshold`	loss_fp / (loss_fp + loss_fn)
`decision`	treat / do not treat (if p given)

- **Method:** equal expected losses
- **Assumes:** losses > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('treatment_threshold', loss_fp=1, loss_fn=9, probability=0.15)
# -> threshold = 0.1, decision = treat   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('treatment_threshold', loss_fp=20, loss_fn=80)
# -> threshold = 0.2, decision = none   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Treatment threshold with loss_fp = 1 and loss_fn = 9; the probability of disease is 0.05. Treat?	SOLVED	threshold = 0.1, decision = do not treat
Decision threshold when a false-positive loss is 20 and a false-negative loss is 80.	SOLVED	threshold = 0.2, decision = none
Treatment threshold with loss_fp = -1 and loss_fn = 9.	INVALID_INPUT	Decision threshold probability (treat / do not treat): the loss of a false positive must be positive.

Uncertainty of a sum or linear combination (independent inputs) — `uncertainty_sum`

Value and combined standard uncertainty of y = sum c_i x_i for independent inputs (default $c_i = 1$; use -1 for a difference).

$$u_y = \sqrt{\sum_i c_i^2 u_i^2}$$

input	meaning	condition
`values`	input values x_i	
`uncertainties`	standard uncertainties u_i (>= 0)	
`coefficients`	coefficients c_i (default all 1)	

output	meaning
`value`	y = sum c_i x_i
`uncertainty`	$u_y = sqrt$(sum (c_i u_i)^2)

- **Method:** root-sum-of-squares (GUM, first order)
- **Assumes:** independent inputs; $u_i \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('uncertainty_sum', values='10, 4', uncertainties='0.3, 0.4',
coefficients='1, -1')
# -> value = 6, uncertainty = 0.5   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('uncertainty_sum', values=[12.3, 4.5, 7.1], uncertainties=[0.2,
0.1, 0.3])
# -> value = 23.9, uncertainty = 0.374166   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Uncertainty of a sum: values = [12.3, 4.5, 7.1], uncertainties = [0.2, 0.1, 0.3].	SOLVED	value = 23.9, uncertainty = 0.374166
Linear combination with values [10, 4], uncertainties [0.3, 0.4] and coefficients [1, -1]: uncertainty of the difference?	SOLVED	value = 6, uncertainty = 0.5
Uncertainty of a sum: values = [12.3, 4.5], uncertainties = [0.2, -0.1].	INVALID_INPUT	Uncertainty of a sum or linear combination (independent inputs): standard uncertainties must not be negative.

Uncertainty of a product and a quotient — `uncertainty_product_quotient`

Value and first-order standard uncertainty of ab and a/b for independent a and b.

$$\frac{u_{ab}}{|ab|} = \frac{u_{a/b}}{|a/b|} = \sqrt{\left(\frac{u_a}{a}\right)^2 + \left(\frac{u_b}{b}\right)^2}$$

input	meaning	condition
a	value of a	
ua	standard uncertainty of a (>= 0)	

input	meaning	condition
`b`	value of b (non-zero)	
`ub`	standard uncertainty of b (>= 0)	

output	meaning
`product`	ab
`u_product`	u(a*b)
`quotient`	a/b
`u_quotient`	u(a/b)

- **Method:** first-order propagation (relative uncertainties add in quadrature)
- **Assumes:** independent a and b; $b \neq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('uncertainty_product_quotient', a=12.0, ua=0.3, b=4.0, ub=0.2)
# -> product = 48, u_product = 2.68328, quotient = 3, u_quotient = 0.167705   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('uncertainty_product_quotient', a=5, ua=0.1, b=2, ub=0.05)
# -> product = 10, u_product = 0.320156, quotient = 2.5, u_quotient = 0.0800391   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Uncertainty of the product and quotient: a = 12.0, ua = 0.3, b = 4.0, ub = 0.2.	SOLVED	product = 48, u_product = 2.68328, quotient = 3, u_quotient = 0.167705
Find the uncertainty of the product of a = 5 with u(a) = 0.1 and b = 2 with u(b) = 0.05.	SOLVED	product = 10, u_product = 0.320156, quotient = 2.5, u_quotient = 0.0800391
Uncertainty of the product and quotient: a = 12.0, ua = 0.3, b = 0, ub = 0.2.	INVALID_INPUT	Uncertainty of a product and a quotient: b must not be 0 (the quotient a/b).

Uncertainty of a product and quotient with correlation — `uncertainty_product_correlated`

First-order standard uncertainty of ab and a/b when a and b are correlated with coefficient rho.

$$u_{ab}^2 = b^2 u_a^2 + a^2 u_b^2 + 2ab\,\rho\,u_a u_b$$

input	meaning	condition
`a`	value of a	
`ua`	standard uncertainty of a	
`b`	value of b (non-zero)	
`ub`	standard uncertainty of b	
`rho`	correlation coefficient of a and b (-1 to 1)	

output	meaning
`product`	ab

output	meaning
`u_product`	u(a*b)
`quotient`	a/b
`u_quotient`	u(a/b)

- **Method:** first-order propagation with the covariance term
- **Assumes:** $-1 \le \rho \le 1$; $b \neq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('uncertainty_product_correlated', a=1, ua=0.1, b=1, ub=0.1, rho=1)
# -> product = 1, u_product = 0.2, quotient = 1, u_quotient = 0   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('uncertainty_product_correlated', a=12.0, ua=0.3, b=4.0, ub=0.2,
rho=0.5)
# -> product = 48, u_product = 3.1749, quotient = 3, u_quotient = 0.129904   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Uncertainty of the product with correlation: a = 1, ua = 0.1, b = 1, ub = 0.1, rho = -1.	SOLVED	product = 1, u_product = 0, quotient = 1, u_quotient = 0.2
Uncertainty of the product with correlation: a = 1, ua = 0.1, b = 1, ub = 0.1, rho = 1.5.	INVALID_INPUT	Uncertainty of a product and quotient with correlation: a correlation coefficient lies between -1 and 1.

Uncertainty of a power x^n — `uncertainty_power`

Value, standard uncertainty and relative uncertainty of $y = x^n$ (first order).

$$\frac{u_y}{|y|} = |n| \, \frac{u_x}{|x|}$$

input	meaning	condition
`x`	value of x	
`ux`	standard uncertainty of x (>= 0)	
`exponent`	exponent n	

output	meaning
`value`	x^n
`uncertainty`	|n x^{n-1}| u(x)
`relative_uncertainty`	|n| u(x)/|x|

- **Method:** first-order propagation
- **Assumes:** $x > 0$ for a non-integer exponent; $x \neq 0$ for $n < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('uncertainty_power', x=3.0, ux=0.1, exponent=2)
# -> value = 9, uncertainty = 0.6, relative_uncertainty = 0.0666667   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('uncertainty_power', x=16, ux=0.4, exponent=0.5)
# -> value = 4, uncertainty = 0.05, relative_uncertainty = 0.0125   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Uncertainty of a power: x = 3.0, ux = 0.1, exponent = 2.	SOLVED	value = 9, uncertainty = 0.6, relative_uncertainty = 0.0666667
Find the uncertainty of x^n with x = 16, u(x) = 0.4 and n = 0.5.	SOLVED	value = 4, uncertainty = 0.05, relative_uncertainty = 0.0125
Uncertainty of a power: x = -3.0, ux = 0.1, exponent = 0.5.	INVALID_INPUT	Uncertainty of a power x^n: a negative x needs a whole-number exponent.

Uncertainty of a function of one variable — `uncertainty_function`

Value and first-order standard uncertainty of f(x) for exp, ln, log10, sqrt, sin, cos, tan (x in radians) or $1/x$.

$$u_f = |f'(x)|u_x$$

input	meaning	condition
`x`	value of x (radians for sin, cos, tan)	
`ux`	standard uncertainty of x	
`function`	exp, ln, log10, sqrt, sin, cos, tan or reciprocal	

output	meaning
`value`	f(x)
`uncertainty`	|f'(x)| u(x)

- **Method:** first-order propagation u_f = |f'(x)| u_x
- **Assumes:** x inside the function's domain
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('uncertainty_function', x=2.5, ux=0.05, function='ln')
# -> value = 0.916291, uncertainty = 0.02   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('uncertainty_function', x=0.0, ux=0.01, function='exp')
# -> value = 1, uncertainty = 0.01   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Uncertainty of a function: function = ln, x = 2.5, ux = 0.05.	SOLVED	value = 0.916291, uncertainty = 0.02

question	outcome	result
Uncertainty of a function: function = sqrt, x = 4, ux = 0.2.	SOLVED	value = 2, uncertainty = 0.05
Uncertainty of a function: function = ln, x = -2.5, ux = 0.05.	INVALID_INPUT	Uncertainty of a function of one variable: ln needs x > 0.

Uncertainty of a power-law product y = k x1^p1 x2^p2 ... — `uncertainty_power_law`

Value and first-order uncertainty of a product of powers of independent positive inputs (relative uncertainties weighted by the exponents add in quadrature).

$$\frac{u_y}{|y|} = \sqrt{\sum_i \left(p_i \, \frac{u_i}{x_i} \right)^2}$$

input	meaning	condition
`coefficient`	exact constant k	
`values`	positive input values x_i	
`exponents`	exponents p_i	
`uncertainties`	standard uncertainties u_i	

output	meaning
`value`	y
`uncertainty`	u_y
`relative_uncertainty`	u_y/|y|

- **Method:** first-order propagation in relative form
- **Assumes:** $x_i > 0$; independent inputs
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('uncertainty_power_law', coefficient=0.5, values='2, 3',
exponents='1, 2', uncertainties='0.02, 0.06')
# -> value = 9, uncertainty = 0.37108, relative_uncertainty = 0.0412311   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('uncertainty_power_law', coefficient=1, values=[10, 4], exponents=
[2, -1], uncertainties=[0.1, 0.08])
# -> value = 25, uncertainty = 0.707107, relative_uncertainty = 0.0282843   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Uncertainty of a power law: coefficient = 0.5, values = [2.0, 3.0], exponents = [1, 2], uncertainties = [0.02, 0.06].	SOLVED	value = 9, uncertainty = 0.37108, relative_uncertainty = 0.0412311

question	outcome	result
Uncertainty of a power law: coefficient = 1, values = [-2.0, 3.0], exponents = [1, 2], uncertainties = [0.02, 0.06].	INVALID_INPUT	Uncertainty of a power-law product y = k x1^p1 x2^p2 ...: a power law needs positive values.

Inverse-variance weighted mean of measurements — `inverse_variance_weighted_mean`

Combine measurements of one quantity with different standard uncertainties: weighted mean, its uncertainty, chi-square of consistency and Birge ratio (equal to the posterior under a flat prior).

$$\bar{x}_w = \frac{\sum x_i/u_i^2}{\sum 1/u_i^2}, \quad u(\bar{x}_w) = \left(\sum 1/u_i^2\right)^{-1/2}$$

input	meaning	condition
`values`	measured values	
`uncertainties`	their standard uncertainties (> 0)	

output	meaning
`weighted_mean`	sum(x_i/u_i^2)/sum(1/u_i^2)
`uncertainty`	$1/sqrt$(sum 1/u_i^2)
`chi_square`	sum((x_i - mean)^2/u_i^2)
`birge_ratio`	sqrt($\chi^2/(n-1)$)

- **Method:** inverse-variance weighting
- **Assumes:** $u_i > 0$; $n \geq 2$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('inverse_variance_weighted_mean', values='5.1, 4.9',
uncertainties='0.1, 0.1')
# -> weighted_mean = 5, uncertainty = 0.0707107, chi_square = 2, birge_ratio = 1.41421   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('inverse_variance_weighted_mean', values=[10.2, 9.8, 10.5],
uncertainties=[0.2, 0.1, 0.4])
# -> weighted_mean = 9.90952, uncertainty = 0.0872872, chi_square = 5.4881, birge_ratio = 1.65652
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Combine the measurements [5.1, 4.9] with uncertainties [0.1, 0.1]: weighted mean?	SOLVED	weighted_mean = 5, uncertainty = 0.0707107, chi_square = 2, birge_ratio = 1.41421
Inverse-variance weighted mean of values = [10.2, 9.8] with uncertainties = [0.2, 0].	INVALID_INPUT	Inverse-variance weighted mean of measurements: uncertainties must be positive.

Uncertainty of a linear combination of correlated inputs — `uncertainty_linear_correlated`

Combined standard uncertainty of y = sum c_i x_i when the inputs are correlated (correlation matrix), u_y^2 = sum_ij c_i c_j u_i u_j r_ij.

$$u_y^2 = \sum_i \sum_j c_i c_j u_i u_j r_{ij}$$

input	meaning	condition
`values`	input values	
`uncertainties`	standard uncertainties	
`correlation`	correlation matrix (ones on the diagonal, positive semi-definite)	
`coefficients`	coefficients c_i (default 1)	

output	meaning
`value`	y
`uncertainty`	u_y

- **Method:** quadratic form c^T Sigma c (GUM eq. 13)
- **Assumes:** valid correlation matrix
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('uncertainty_linear_correlated', values='10, 20',
uncertainties='0.3, 0.4', correlation='[[1, 0.5], [0.5, 1]]', coefficients='1, 1')
# -> value = 30, uncertainty = 0.608276   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('uncertainty_linear_correlated', values=[1, 2], uncertainties=
[0.1, 0.1], correlation=[[1, 1], [1, 1]], coefficients=[1, -1])
# -> value = -1, uncertainty = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Uncertainty of a linear combination with correlation matrix [[1, 0.5], [0.5, 1]], values = [10, 20], uncertainties = [0.3, 0.4], coefficients = [1, 1].	SOLVED	value = 30, uncertainty = 0.608276
Uncertainty of a linear combination with correlation matrix [[1, 1.5], [1.5, 1]], values = [10, 20], uncertainties = [0.3, 0.4].	INVALID_INPUT	Uncertainty of a linear combination of correlated inputs: a correlation matrix has ones on the diagonal and entries betw

Covariance propagation through a linear map y = A x — `covariance_propagation_linear`

Covariance matrix of the outputs $y = A$ x and their standard uncertainties, given the covariance matrix of x (first-order propagation with a sensitivity matrix).

$$\Sigma_y = A\,\Sigma_x\,A^{\mathsf{T}}$$

input	meaning	condition
`matrix`	sensitivity (Jacobian) matrix A: one row per output	
`covariance`	covariance matrix of the inputs (symmetric, positive semi-definite)	

output	meaning
`output_covariance`	A Sigma A^T
`uncertainties`	square roots of its diagonal

- **Method:** matrix product A Sigma A^T
- **Assumes:** covariance symmetric positive semi-definite; columns of A = inputs
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('covariance_propagation_linear', matrix='[[1, 1], [1, -1]]',
covariance='[[0.04, 0.01], [0.01, 0.09]]')
# -> output_covariance = [[0.15, -0.05], [-0.05, 0.11]], uncertainties = [0.387298334621,
0.331662479036]   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('covariance_propagation_linear', matrix=[[2, 1]], covariance=[[1,
0], [0, 4]])
# -> output_covariance = [[8.0]], uncertainties = [2.82842712475]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Covariance propagation: matrix = [[1, 1], [1, -1]], covariance = [[0.04, 0.01], [0.01, 0.09]].	SOLVED	output_covariance = [[0.15, -0.05], [-0.05, 0.11]], uncertainties = [0.387298334621, 0.331662479036]
Propagate the covariance matrix [[1, 2], [2, 1]] through the sensitivity matrix [[2, 1]].	INVALID_INPUT	Covariance propagation through a linear map y = A x: the covariance matrix must be positive semi-definite.

Effective degrees of freedom and expanded uncertainty (Welch-Satterthwaite) — `welch_satterthwaite_expanded_uncertainty`

Combined standard uncertainty, Welch-Satterthwaite effective degrees of freedom, the Student-t coverage factor and the expanded uncertainty at a stated coverage probability.

$$\nu_{eff} = \frac{u_c^4}{\sum_i u_i^4/\nu_i}, \quad U = t_{(1+p)/2,\,\nu_{eff}}\, u_c$$

input	meaning	condition
`uncertainties`	uncertainty components c_i u_i (> 0)	
`dof`	degrees of freedom of each component (> 0)	
`level`	credibility level as a fraction (0.95 for 95%; default 0.95)	

output	meaning
`combined_uncertainty`	u_c
`effective_dof`	nu_eff

output	meaning
`coverage_factor`	Missing superscript or subscript argument{$(1+p)/2$, nu_eff}
`expanded_uncertainty`	$U = k$ u_c

- **Method:** Welch-Satterthwaite formula and the Student-t quantile at non-integer degrees of freedom
- **Assumes:** components and degrees of freedom > 0; 0 < level < 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('welch_satterthwaite_expanded_uncertainty', uncertainties='1',
dof='10', level=0.95)
# -> combined_uncertainty = 1, effective_dof = 10, coverage_factor = 2.22814, expanded_uncertainty
= 2.22814   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('welch_satterthwaite_expanded_uncertainty', uncertainties=[0.3,
0.2], dof=[4, 9])
# -> combined_uncertainty = 0.360555, effective_dof = 7.67213, coverage_factor = 2.32327,
expanded_uncertainty = 0.837667   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Welch-Satterthwaite: uncertainties = [1], degrees of freedom = [10]. Expanded uncertainty at 95% coverage?	SOLVED	combined_uncertainty = 1, effective_dof = 10, coverage_factor = 2.22814, expanded_uncertainty = 2.22814
Expanded uncertainty with components [0.1, 0.1], dof = [5, 0] and level = 0.95.	INVALID_INPUT	Effective degrees of freedom and expanded uncertainty (Welch-Satterthwaite): uncertainty components and degrees of freed

Type B standard uncertainty from a stated half-width — `type_b_standard_uncertainty`

Standard uncertainty of a quantity known only to lie within +/- a, for a rectangular, triangular or U-shaped (arcsine) distribution.

$$u = \frac{a}{\sqrt{3}}\ (\text{rect.}),\ \frac{a}{\sqrt{6}}\ (\text{tri.}),\ \frac{a}{\sqrt{2}}\ (\text{arcsine})$$

input	meaning	condition
`half_width`	half-width a of the interval (> 0)	
`distribution`	rectangular, triangular or arcsine (U-shaped)	

output	meaning
`standard_uncertainty`	a / divisor
`divisor`	sqrt(3), sqrt(6) or sqrt(2)

- **Method:** standard deviation of the assumed distribution
- **Assumes:** $a > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('type_b_standard_uncertainty', half_width=0.5,
distribution='rectangular')
# -> standard_uncertainty = 0.288675, divisor = 1.73205   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('type_b_standard_uncertainty', half_width=0.3,
distribution='triangular')
# -> standard_uncertainty = 0.122474, divisor = 2.44949   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Type B uncertainty: half-width = 0.5 with a rectangular distribution.	SOLVED	standard_uncertainty = 0.288675, divisor = 1.73205
Type B uncertainty: half-width = 2 with a U-shaped distribution.	SOLVED	standard_uncertainty = 1.41421, divisor = 1.41421
Type B uncertainty: half-width = -0.5 with a rectangular distribution.	INVALID_INPUT	Type B standard uncertainty from a stated half-width: the half-width must be positive.

Exact versus first-order propagation through exp(X) — `lognormal_propagation`

Mean and standard deviation of $Y = exp$(X) for normal X ~ N(mu, sigma^2), exactly (lognormal) and by first-order propagation, showing the bias of the linear approximation.

$$E[e^X] = e^{\mu+\sigma^2/2}, \quad \mathrm{SD}[e^X] = e^{\mu+\sigma^2/2}\sqrt{e^{\sigma^2}-1}$$

input	meaning	condition
`mu`	mean of X	
`sigma`	standard deviation of X (> 0)	

output	meaning
`exact_mean`	exp(mu + sigma^2/2)
`exact_sd`	sqrt(exp(sigma^2) - 1) exp(mu + sigma^2/2)
`first_order_mean`	exp(mu)
`first_order_sd`	exp(mu) sigma

- **Method:** lognormal moments; first-order Taylor propagation
- **Assumes:** $\sigma > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('lognormal_propagation', mu=0, sigma=1)
# -> exact_mean = 1.64872, exact_sd = 2.1612, first_order_mean = 1, first_order_sd = 1   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('lognormal_propagation', mu=1, sigma=0.5)
# -> exact_mean = 3.08022, exact_sd = 1.64157, first_order_mean = 2.71828, first_order_sd = 1.35914
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Lognormal propagation with mu = 0 and sigma = 1.	SOLVED	exact_mean = 1.64872, exact_sd = 2.1612, first_order_mean = 1, first_order_sd = 1
Lognormal propagation with mu = 0 and sigma = -1.	INVALID_INPUT	Exact versus first-order propagation through exp(X): sigma must be positive.

Exact mean and standard deviation of a product of independent normals — `product_of_normals_exact_moments`

Exact moments of XY for independent normal X and Y, compared with first-order propagation (which misses the sd_x sd_y term).

$$\mathrm{Var}(XY) = \mu_x^2\sigma_y^2 + \mu_y^2\sigma_x^2 + \sigma_x^2\sigma_y^2$$

input	meaning	condition
`mean_x`	mean of X	
`sd_x`	sd of X (> 0)	
`mean_y`	mean of Y	
`sd_y`	sd of Y (> 0)	

output	meaning
`exact_mean`	mean_x mean_y
`exact_sd`	sqrt($mx^2\ sy^2 + my^2\ sx^2 + sx^2$ sy^2)
`first_order_sd`	sqrt($mx^2\ sy^2 + my^2$ sx^2)

- **Method:** exact second moments of a product
- **Assumes:** independent normal inputs
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('product_of_normals_exact_moments', mean_x=0, sd_x=1, mean_y=0, sd_y=1)
# -> exact_mean = 0, exact_sd = 1, first_order_sd = 0   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('product_of_normals_exact_moments', mean_x=10, sd_x=1, mean_y=5, sd_y=2)
# -> exact_mean = 50, exact_sd = 20.7123, first_order_sd = 20.6155   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Exact moments of the product of independent normals: mean_x = 10, sd_x = 1, mean_y = 5, sd_y = 2.	SOLVED	exact_mean = 50, exact_sd = 20.7123, first_order_sd = 20.6155

question	outcome	result
Exact moments of the product of independent normals: mean_x = 10, sd_x = 0, mean_y = 5, sd_y = 2.	INVALID_INPUT	Exact mean and standard deviation of a product of independent normals: sd_x must be positive.

Number of Monte Carlo draws for a target accuracy — `monte_carlo_sample_size`

How many independent draws make the half-width of the confidence interval of a Monte Carlo mean at most the stated value, given the standard deviation of one draw.

$$n = \left\lceil \left(\frac{z_{(1+c)/2}\,\sigma}{E} \right)^2 \right\rceil$$

input	meaning	condition
`sigma`	standard deviation of one draw (> 0)	
`half_width`	target half-width (> 0)	
`level`	credibility level as a fraction (0.95 for 95%; default 0.95)	

output	meaning
`draws`	required number of draws (rounded up)
`z`	normal quantile z_{$(1+c)/2$}
`standard_error_target`	target standard error half_width/z

- **Method:** n = (z sigma / E)^2, rounded up
- **Assumes:** sigma, half-width > 0; 0 < level < 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('monte_carlo_sample_size', sigma=10, half_width=0.5, level=0.95)
# -> draws = 1537, z = 1.95996, standard_error_target = 0.255107   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('monte_carlo_sample_size', sigma=1, half_width=0.1, level=0.9)
# -> draws = 271, z = 1.64485, standard_error_target = 0.0607957   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Monte Carlo sample size: standard deviation 10, half-width 0.5, 95% confidence. How many draws?	SOLVED	draws = 1537, z = 1.95996, standard_error_target = 0.255107
Monte Carlo sample size: standard deviation 10, half-width 0, 95% confidence.	INVALID_INPUT	Number of Monte Carlo draws for a target accuracy: the half-width must be positive.

Monte Carlo estimate of an integral from uniform samples — `monte_carlo_integration`

Estimate of the integral of f over [a, b] from f evaluated at n points drawn uniformly on [a, b], with its Monte Carlo standard error.

$$\hat{I} = \frac{b-a}{n}\sum_i f(x_i), \quad SE = (b-a)\frac{s_f}{\sqrt{n}}$$

input	meaning	condition
`values`	f(x_i) at uniform sample points	
`lower`	lower limit a	
`upper`	upper limit b (> a)	

output	meaning
`estimate`	(b - a) times the mean of f
`standard_error`	(b - a) s / sqrt(n)
`n`	number of samples

- **Method:** sample mean of f scaled by the interval length
- **Assumes:** $b > a$; $n \geq 2$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('monte_carlo_integration', values='1, 3', lower=0, upper=2)
# -> estimate = 4, standard_error = 2, n = 2   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('monte_carlo_integration', values=[3, 5, 4, 6], lower=1, upper=4)
# -> estimate = 13.5, standard_error = 1.93649, n = 4   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Monte Carlo integration with values = [1, 3]: lower = 0, upper = 2.	SOLVED	estimate = 4, standard_error = 2, n = 2
Monte Carlo integration with values = [1, 3]: lower = 2, upper = 0.	INVALID_INPUT	Monte Carlo estimate of an integral from uniform samples: the upper limit must exceed the lower limit.

Importance sampling estimate and effective sample size — `importance_sampling_estimate`

Self-normalized and plain importance-sampling estimates of E_p[f] from draws of a proposal q, with the effective sample size of the weights p/q.

$$\hat{\mu} = \frac{\sum_i w_i f(x_i)}{\sum_i w_i}, \; w_i = \frac{p(x_i)}{q(x_i)}, \quad ESS = \frac{(\sum w_i)^2}{\sum w_i^2}$$

input	meaning	condition
`values`	f(x_i) at the draws	
`target_density`	target density p(x_i) (or unnormalized)	
`proposal_density`	proposal density q(x_i) > 0	

output	meaning
`estimate`	self-normalized estimate sum w f / sum w
`unnormalized_estimate`	mean of w f (needs normalized densities)
`effective_sample_size`	(sum w)^2 / sum w^2

- **Method:** weights $w = p/q$
- **Assumes:** $q > 0$; $p \geq 0$, not all zero
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('importance_sampling_estimate', values='0, 1',
target_density='0.5, 0.5', proposal_density='0.25, 0.75')
# -> estimate = 0.25, unnormalized_estimate = 0.333333, effective_sample_size = 1.6   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('importance_sampling_estimate', values=[1, 2, 3], target_density=
[0.2, 0.5, 0.3], proposal_density=[0.4, 0.4, 0.2])
# -> estimate = 2.30769, unnormalized_estimate = 2.5, effective_sample_size = 2.6   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Importance sampling estimate with f values [0, 1], target density [0.5, 0.5] and proposal density [0.25, 0.75].	SOLVED	estimate = 0.25, unnormalized_estimate = 0.333333, effective_sample_size = 1.6
Importance sampling: values = [1, 2], target densities = [0.2, 0.5], proposal densities = [0.4, 0].	INVALID_INPUT	Importance sampling estimate and effective sample size: proposal densities must be positive and target densities non-neg

Control-variate Monte Carlo estimate — `control_variate_estimate`

Variance-reduced estimate of E[Y] from paired draws (y_i, c_i) of the output and a control variate with known mean: optimal coefficient, estimate and the fraction of variance left $(1 - r^2)$.

$$\hat{\mu}_{cv} = \bar{y} - \hat{b}\,(\bar{c} - \mu_c), \quad \hat{b} = \frac{\sum (y_i - \bar{y})(c_i - \bar{c})}{\sum (c_i - \bar{c})^2}$$

input	meaning	condition
`y`	simulated outputs y_i	
`c`	control values c_i (same draws)	
`control_mean`	known mean of the control	

output	meaning
`estimate`	mean(y) - b (mean(c) - mu_c)
`coefficient`	b = cov(y, c)/var(c)
`plain_estimate`	mean(y)
`variance_reduction`	variance factor $1 - r^2$

- **Method:** regression (optimal) control-variate coefficient
- **Assumes:** outputs and control vary; $n \geq 3$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('control_variate_estimate', y='1, 2, 3', c='1, 2, 3',
control_mean=2.5)
# -> estimate = 2.5, coefficient = 1, plain_estimate = 2, variance_reduction = 0   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('control_variate_estimate', y=[2.1, 3.9, 6.2, 8.1], c=[1, 2, 3,
4], control_mean=2.5)
# -> estimate = 5.075, coefficient = 2.03, plain_estimate = 5.075, variance_reduction = 0.00208258
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Control variate: outputs = [1, 2, 3], controls = [1, 2, 3], known mean = 2.5.	SOLVED	estimate = 2.5, coefficient = 1, plain_estimate = 2, variance_reduction = 0
Control variate: outputs = [1, 2, 3], controls = [1, 1, 1], known mean = 2.5.	INVALID_INPUT	Control-variate Monte Carlo estimate: the outputs and the control must both vary.

Stratified sampling estimate and standard error — `stratified_sampling_estimate`

Estimate of a mean from strata with known weights, the sample mean, standard deviation and size of each stratum, and its standard error.

$$\hat{\mu} = \sum_h W_h \bar{y}_h, \quad SE = \sqrt{\sum_h W_h^2 s_h^2 / n_h}$$

input	meaning	condition
`weights`	stratum weights (sum to 1)	
`means`	stratum sample means	
`sds`	stratum sample standard deviations	
`sizes`	stratum sample sizes	

output	meaning
`estimate`	sum W_h mean_h
`standard_error`	sqrt(sum W_h^2 s_h^2 / n_h)

- **Method:** weighted stratum means
- **Assumes:** weights sum to 1; sizes >= 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('stratified_sampling_estimate', weights='0.6, 0.4', means='10,
20', sds='2, 5', sizes='30, 20')
# -> estimate = 14, standard_error = 0.497996   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('stratified_sampling_estimate', weights=[0.5, 0.5], means=[1, 3],
sds=[1, 1], sizes=[4, 4])
# -> estimate = 2, standard_error = 0.353553   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Stratified sampling: weights = [0.6, 0.4], means = [10, 20], sds = [2, 5], sizes = [30, 20].	SOLVED	estimate = 14, standard_error = 0.497996
Stratified sampling: weights = [0.6, 0.6], means = [10, 20], sds = [2, 5], sizes = [30, 20].	INVALID_INPUT	Stratified sampling estimate and standard error: the stratum weights must sum to 1 (they sum to 1.2).

Point of the Halton low-discrepancy sequence — `halton_point`

Coordinates of the point with a given index in the (unscrambled) Halton sequence of a given dimension: the radical inverses of the index in the first prime bases (index 0 is the origin).

$$\phi_b(i) = \sum_k d_k\, b^{-k-1} \text{ for } i = \sum_k d_k b^k$$

input	meaning	condition
`index`	index of the point (0, 1, 2, ...)	
`dimension`	dimension (1 to 20)	

output	meaning
`point`	coordinates in [0, 1)
`bases`	prime bases used

- **Method:** radical inverse: reverse the base-b digits of the index behind the radix point
- **Assumes:** index >= 0; 1 <= dimension <= 20
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('halton_point', index=5, dimension=3)
# -> point = [0.625, 0.777777777778, 0.04], bases = [2.0, 3.0, 5.0]   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('halton_point', index=1, dimension=2)
# -> point = [0.5, 0.333333333333], bases = [2.0, 3.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Halton sequence point with index = 5 and dimension = 3.	SOLVED	point = [0.625, 0.777777777778, 0.04], bases = [2.0, 3.0, 5.0]
Halton sequence point with index = 5 and dimension = 30.	INVALID_INPUT	Point of the Halton low-discrepancy sequence: dimension must be at most 20.

Sobol sensitivity indices of the Ishigami function — `sobol_indices_ishigami`

Exact first-order and total Sobol indices and the variance of the Ishigami test function $f = sin\, x_1 + a\, sin^2\, x_2 + b\, x_3^4$ sin x1 with x_i uniform on [-pi, pi].

$$V = \frac{a^2}{8} + \frac{b\pi^4}{5} + \frac{b^2\pi^8}{18} + \frac{1}{2},\ V_1 = \frac{1}{2}\left(1 + \frac{b\pi^4}{5}\right)^2,\ V_2 = \frac{a^2}{8},\ V_{13} = \frac{8b^2\pi^8}{225}$$

input	meaning	condition
`a`	parameter a (commonly 7)	
`b`	parameter b (commonly 0.1)	

output	meaning
`S1`	first-order index of x1
`S2`	first-order index of x2
`S3`	first-order index of x3
`ST1`	total index of x1
`ST2`	total index of x2
`ST3`	total index of x3
`variance`	variance of f

- **Method:** closed-form variance decomposition
- **Assumes:** inputs independent uniform on [-pi, pi]
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('sobol_indices_ishigami', a=7, b=0.1)
# -> S1 = 0.313905, S2 = 0.442411, S3 = 0, ST1 = 0.557589, ST2 = 0.442411, ST3 = 0.243684, variance
= 13.8446   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('sobol_indices_ishigami', a=0, b=0)
# -> S1 = 1, S2 = 0, S3 = 0, ST1 = 1, ST2 = 0, ST3 = 0, variance = 0.5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Sobol indices of the Ishigami function with a = 7 and b = 0.1.	SOLVED	S1 = 0.313905, S2 = 0.442411, S3 = 0, ST1 = 0.557589, ST2 = 0.442411, ST3 = 0.243684, variance = 13.8446
Sobol indices of the Ishigami function with a = 7.	NEEDS_INPUT	sobol_indices_ishigami needs b (parameter b (commonly 0.1)). Example: solve('sobol_indices_ishigami', a=7, b=0.1).

Sobol indices of Sobol's G-function — `sobol_indices_g_function`

Exact first-order and total Sobol indices and the variance of the G-function prod_i (|4 x_i − 2| + a_i)/(1 + a_i), x_i uniform on [0, 1].

$$V_i = \frac{1}{3(1+a_i)^2},\ V = \prod_i (1+V_i) - 1,\ S_i = \frac{V_i}{V},\ S_{Ti} = \frac{V_i \prod_{j \neq i}(1+V_j)}{V}$$

input	meaning	condition
`a`	parameters $a_i \geq 0$ (small a_i = important input)	

output	meaning
`first_order`	first-order indices S_i
`total_order`	total indices ST_i
`variance`	variance of the function

- **Method:** partial variances V_i = 1/(3(1 + a_i)^2) of the product form
- **Assumes:** $a_i \geq 0$; independent uniform inputs
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('sobol_indices_g_function', a='0')
# -> first_order = [1.0], total_order = [1.0], variance = 0.333333   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('sobol_indices_g_function', a=[0, 0])
# -> first_order = [0.428571428571, 0.428571428571], total_order = [0.571428571429,
0.571428571429], variance = 0.777778   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Sobol indices of the G-function with a = [0, 0].	SOLVED	first_order = [0.428571428571, 0.428571428571], total_order = [0.571428571429, 0.571428571429], variance = 0.777778
Sobol indices of the G-function with a = [0, -1].	INVALID_INPUT	Sobol indices of Sobol's G-function: the G-function parameters must be >= 0.

Sobol indices of a linear (additive) model — `sobol_indices_linear_model`

First-order Sobol indices of y = sum a_i x_i with independent inputs of given standard deviations (S_i = a_i^2 sigma_i^2 / Var y; they sum to 1).

$$S_i = \frac{a_i^2 \sigma_i^2}{\sum_j a_j^2 \sigma_j^2}$$

input	meaning	condition
`coefficients`	model coefficients a_i	
`sds`	input standard deviations sigma_i (> 0)	

output	meaning
`first_order`	first-order (= total) indices
`variance`	Var y

- **Method:** variance shares of the terms
- **Assumes:** independent inputs; $sigma_i > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('sobol_indices_linear_model', coefficients='1, 1', sds='3, 4')
# -> first_order = [0.36, 0.64], variance = 25   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('sobol_indices_linear_model', coefficients=[2, 1, 0.5], sds=[1, 2, 4])
# -> first_order = [0.333333333333, 0.333333333333, 0.333333333333], variance = 12   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Sobol indices of a linear model with coefficients = [2, 1, 0.5] and sds = [1, 2, 4].	SOLVED	first_order = [0.333333333333, 0.333333333333, 0.333333333333], variance = 12
Sobol indices of a linear model with coefficients = [2, 1] and sds = [1, 0].	INVALID_INPUT	Sobol indices of a linear (additive) model: input standard deviations must be positive.

Sobol indices of a model with an interaction term — `sobol_indices_interaction_model`

First-order, interaction and total Sobol indices of $y = a_1\,x_1 + a_2\,x_2 + a12$ x1 x2 with independent normal inputs x1 ~ N(mean1, sd1^2), x2 ~ N(mean2, sd2^2).

$$V_1 = (a_1 + a_{12}\mu_2)^2\sigma_1^2,\; V_2 = (a_2 + a_{12}\mu_1)^2\sigma_2^2,\; V_{12} = a_{12}^2\sigma_1^2\sigma_2^2$$

input	meaning	condition
`a1`	coefficient of x1	
`a2`	coefficient of x2	
`a12`	interaction coefficient	
`mean1`	mean of x1	
`sd1`	sd of x1 (> 0)	
`mean2`	mean of x2	
`sd2`	sd of x2 (> 0)	

output	meaning
`S1`	first-order index of x1
`S2`	first-order index of x2
`S12`	interaction index
`ST1`	total index of x1
`ST2`	total index of x2
`variance`	Var y

- **Method:** ANOVA decomposition of the bilinear polynomial
- **Assumes:** independent normal inputs

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('sobol_indices_interaction_model', a1=0, a2=0, a12=1, mean1=0,
sd1=1, mean2=0, sd2=1)
# -> S1 = 0, S2 = 0, S12 = 1, ST1 = 1, ST2 = 1, variance = 1   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('sobol_indices_interaction_model', a1=1, a2=2, a12=0.5, mean1=0,
sd1=1, mean2=1, sd2=0.5)
# -> S1 = 0.679245, S2 = 0.301887, S12 = 0.0188679, ST1 = 0.698113, ST2 = 0.320755, variance =
3.3125   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Sobol indices of the interaction model with a1 = 0, a2 = 0, a12 = 1, mean1 = 0, sd1 = 1, mean2 = 0, sd2 = 1.	SOLVED	S1 = 0, S2 = 0, S12 = 1, ST1 = 1, ST2 = 1, variance = 1
Sobol indices of the interaction model with a1 = 1, a2 = 2, a12 = 0.5, mean1 = 0, sd1 = 0, mean2 = 1, sd2 = 0.5.	INVALID_INPUT	Sobol indices of a model with an interaction term: sd1 must be positive.

Morris elementary-effects screening — `morris_elementary_effects`

Mean, mean absolute value (mu*) and standard deviation of the elementary effects from one-at-a-time trajectories: in each row, output $j+1$ is the model at the previous point with input j raised by delta.

$$EE_j = \frac{f(x+\Delta e_j) - f(x)}{\Delta}, \quad \mu_j^* = \frac{1}{r}\sum_t |EE_j^{(t)}|$$

input	meaning	condition
`outputs`	model outputs: one row per trajectory, $k+1$ values (base point, then input 1, 2, ... raised in turn)	
`delta`	step delta (> 0)	

output	meaning
`mu`	mean elementary effect of each input
`mu_star`	mean absolute elementary effect
`sigma`	standard deviation of the elementary effects

- **Method:** elementary effects EE_ij = (y_{i,$j+1$} - y_{ij})/delta
- **Assumes:** at least 2 trajectories; $\delta > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('morris_elementary_effects', outputs='[[1, 3, 4], [2, 3, 7], [0,
1, 1]]', delta=0.5)
# -> mu = [2.66666666667, 3.33333333333], mu_star = [2.66666666667, 3.33333333333], sigma =
[1.15470053838, 4.16333199893]   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('morris_elementary_effects', outputs=[[10, 12], [11, 10]],
delta=1)
# -> mu = [0.5], mu_star = [1.5], sigma = [2.12132034356]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Elementary effects screening with model outputs [[10, 12], [11, 10]] and step 1.	SOLVED	mu = [0.5], mu_star = [1.5], sigma = [2.12132034356]
Morris elementary effects: outputs = [[1, 3, 4]], delta = 0.5.	INVALID_INPUT	Morris elementary-effects screening: give at least 2 trajectories, each with k + 1 >= 2 outputs.

Kullback-Leibler divergence between two normal distributions — `kl_divergence_normal`

KL(N1 || N2) and KL(N2 || N1), e.g. the information gained from a normal prior to a normal posterior.

$$KL(N_1 \| N_2) = \ln \frac{\sigma_2}{\sigma_1} + \frac{\sigma_1^2 + (\mu_1 - \mu_2)^2}{2\sigma_2^2} - \frac{1}{2}$$

input	meaning	condition
`mean1`	mean of distribution 1	
`sd1`	sd of distribution 1 (> 0)	
`mean2`	mean of distribution 2	
`sd2`	sd of distribution 2 (> 0)	

output	meaning
`kl_1_to_2`	KL(N1 || N2) in nats
`kl_2_to_1`	KL(N2 || N1) in nats

- **Method:** closed form for two normals
- **Assumes:** sd1, $sd2 > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('kl_divergence_normal', mean1=0, sd1=1, mean2=1, sd2=1)
# -> kl_1_to_2 = 0.5, kl_2_to_1 = 0.5   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('kl_divergence_normal', mean1=0, sd1=1, mean2=1, sd2=2)
# -> kl_1_to_2 = 0.443147, kl_2_to_1 = 1.30685   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
KL divergence between normal distributions: mean1 = 0, sd1 = 1, mean2 = 1, sd2 = 2.	SOLVED	kl_1_to_2 = 0.443147, kl_2_to_1 = 1.30685
KL divergence between normal distributions: mean1 = 0, sd1 = 1, mean2 = 1, sd2 = 0.	INVALID_INPUT	Kullback-Leibler divergence between two normal distributions: sd2 must be positive.

Kullback-Leibler divergence between two beta distributions — `kl_divergence_beta`

KL(Beta(alpha1, beta1) || Beta(alpha2, beta2)): for example the information gained from a beta prior (distribution 2) to a beta posterior (distribution 1).

$$KL = \ln \frac{B(\alpha_2, \beta_2)}{B(\alpha_1, \beta_1)} + (\alpha_1 - \alpha_2)\psi(\alpha_1) + (\beta_1 - \beta_2)\psi(\beta_1) + (\alpha_2 - \alpha_1 + \beta_2 - \beta_1)\psi(\alpha_1 + \beta_1)$$

input	meaning	condition
`alpha1`	alpha of distribution 1	
`beta1`	beta of distribution 1	
`alpha2`	alpha of distribution 2	
`beta2`	beta of distribution 2	

output	meaning
`kl`	KL divergence in nats

- **Method:** closed form with log-beta and digamma functions
- **Assumes:** all parameters > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('kl_divergence_beta', alpha1=2, beta1=1, alpha2=1, beta2=1)
# -> kl = 0.193147   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('kl_divergence_beta', alpha1=8, beta1=4, alpha2=1, beta2=1)
# -> kl = 0.636614   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
KL divergence between beta distributions: alpha1 = 2, beta1 = 1, alpha2 = 1, beta2 = 1.	SOLVED	kl = 0.193147
KL divergence between beta distributions: alpha1 = 2, beta1 = 0, alpha2 = 1, beta2 = 1.	INVALID_INPUT	Kullback-Leibler divergence between two beta distributions: beta1 must be positive.

Gauss-Hermite rule for expectations under a normal distribution — `gauss_hermite_rule`

Nodes and weights of the n-point Gauss-Hermite rule for E[f(Z)], Z standard normal (probabilists' Hermite polynomials; weights sum to 1; exact for polynomials of degree up to 2n - 1). For N(mu, sigma^2) use mu + sigma x_i.

$$E[f(Z)] \approx \sum_{i=1}^{n} w_i f(x_i), \quad He_n(x_i) = 0, \quad w_i = \frac{n!}{n^2\, He_{n-1}(x_i)^2}$$

input	meaning	condition
`n`	number of nodes (1 to 40)	

output	meaning
`nodes`	nodes x_i (ascending)
`weights`	weights w_i (sum to 1)

- **Method:** Golub-Welsch: eigenvalues of the Jacobi matrix (implicit QL), Newton-polished on He_n; weights n!/(n^2 He_{$n-1$}(x_i)^2)
- **Assumes:** $1 \le n \le 40$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('gauss_hermite_rule', n=2)
# -> nodes = [-1.0, 1.0], weights = [0.5, 0.5]   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('gauss_hermite_rule', n=3)
# -> nodes = [-1.73205080757, 0.0, 1.73205080757], weights = [0.166666666667, 0.666666666667,
0.166666666667]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Gauss-Hermite rule with n = 3 nodes for a normal expectation.	SOLVED	nodes = [-1.73205080757, 0.0, 1.73205080757], weights = [0.166666666667, 0.666666666667, 0.166666666667]
Gauss-Hermite rule with n = 0 nodes.	INVALID_INPUT	n: must be at least 1, got 0.

Exact mean and variance of a polynomial of a normal variable — `polynomial_of_normal_moments`

Exact mean, variance and standard deviation of $Y = c_0 + c_1\,X + \ldots + c_6\,X^6$ for X ~ N(mean, sd^2) (nonlinear uncertainty propagation without linearization).

$$E[Z^{2m}] = (2m-1)!!, \quad X = \mu + \sigma Z$$

input	meaning	condition
`coefficients`	coefficients c0, c1, ... (constant first, up to degree 6)	
`mean`	mean of X	
`sd`	standard deviation of X (> 0)	

output	meaning
`mean`	E[Y]
`variance`	Var[Y]
`sd`	sd of Y

- **Method:** substitute X = mean + sd Z and use the normal moments E[Z^k] = $(k-1)$!! for even k
- **Assumes:** $sd > 0$; degree <= 6
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('polynomial_of_normal_moments', coefficients='1, 2, 3', mean=0, sd=1)
# -> mean = 4, variance = 22, sd = 4.69042   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('polynomial_of_normal_moments', coefficients=[0, 0, 1], mean=2, sd=0.5)
# -> mean = 4.25, variance = 4.125, sd = 2.03101   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Mean and variance of the polynomial with coefficients = [1, 2, 3] of a normal X with mean = 0, sd = 1.	SOLVED	mean = 4, variance = 22, sd = 4.69042
Mean and variance of the polynomial with coefficients = [1, 2, 3] of a normal X with mean = 0, sd = 0.	INVALID_INPUT	Exact mean and variance of a polynomial of a normal variable: the standard deviation must be positive.

Probability that one Poisson rate exceeds another (gamma posteriors) — `probability_rate_greater_gamma`

P(lambda_1 > lambda_2) for independent Gamma(shape, rate) posteriors of two Poisson rates, and the ratio of their posterior means.

$$P(\lambda_1 > \lambda_2) = 1 - I_{b_1/(b_1+b_2)}(a_1, a_2)$$

input	meaning	condition
`shape1`	posterior shape of rate 1	
`rate1`	posterior rate parameter of rate 1	
`shape2`	posterior shape of rate 2	
`rate2`	posterior rate parameter of rate 2	

output	meaning
`prob_1_greater`	P(lambda_1 > lambda_2)
`prob_2_greater`	P(lambda_2 > lambda_1)
`rate_ratio_mean`	ratio of posterior means

- **Method:** lambda_1 b_1 / (lambda_1 $b_1 + lambda_2$ b_2) ~ Beta(shape1, shape2): one regularized incomplete beta
- **Assumes:** all parameters > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
bayesian_statistics_ai999x.solve('probability_rate_greater_gamma', shape1=5, rate1=1, shape2=5, rate2=1)
# -> prob_1_greater = 0.5, prob_2_greater = 0.5, rate_ratio_mean = 1   (SOLVED)
```

```
bayesian_statistics_ai999x.solve('probability_rate_greater_gamma', shape1=1, rate1=1, shape2=1,
rate2=2)
# -> prob_1_greater = 0.666667, prob_2_greater = 0.333333, rate_ratio_mean = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Compare gamma posteriors shape1 = 5, rate1 = 1, shape2 = 5, rate2 = 1.	SOLVED	prob_1_greater = 0.5, prob_2_greater = 0.5, rate_ratio_mean = 1
Compare gamma posteriors shape1 = 1, rate1 = 1, shape2 = 1, rate2 = 2.	SOLVED	prob_1_greater = 0.666667, prob_2_greater = 0.333333, rate_ratio_mean = 2
Compare gamma posteriors shape1 = 5, rate1 = 0, shape2 = 5, rate2 = 1.	INVALID_INPUT	Probability that one Poisson rate exceeds another (gamma posteriors): rate1 must be positive.